🧠 AI Evolution Program

Parte 01 — IA simbólica, búsqueda, lógica y planificación

Recorre la primera gran tradición de la IA: representar problemas de forma explícita y resolverlos mediante búsqueda, reglas, restricciones y planes.

1. 013 — Espacios de estados y formulación de problemas
2. 014 — Búsqueda en anchura y profundidad
3. 015 — Costo uniforme, búsqueda voraz y A*
4. 016 — Diseño y validación de heurísticas
5. 017 — Juegos: minimax y poda alfa-beta
6. 018 — Problemas de satisfacción de restricciones
7. 019 — Lógica proposicional e inferencia
8. 020 — Lógica de primer orden y unificación
9. 021 — Representación del conocimiento y ontologías
10. 022 — Sistemas expertos y motores de reglas
11. 023 — Planificación clásica con STRIPS y PDDL
12. 024 — Proyecto: asistente neuro-simbólico explicable

013 — Espacios de estados y formulación de problemas

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `search` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **espacios de estados y formulación de problemas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar espacios de estados y formulación de problemas usando los conceptos `estado`, `acciones`, `objetivo`, `costo`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`estado`, `acciones`, `objetivo`, `costo`

🗺️ Ubicación en el mapa de la IA

La formulación de problemas como espacios de estados es la puerta de entrada a la IA simbólica: fue el marco con el que Newell y Simon plantearon el *General Problem Solver* y el que organiza los capítulos de búsqueda de AIMA. Precede a todos los algoritmos de búsqueda (BFS, DFS, A*, minimax) porque ninguno puede ejecutarse sin un problema bien formulado. Habilita después la planificación clásica (STRIPS/PDDL), donde los estados se describen con lógica en lugar de enumerarse, y sigue vigente en aprendizaje por refuerzo, donde el MDP es un espacio de estados con transiciones estocásticas.

📖 Fundamentos

🧩 Los cinco componentes de un problema de búsqueda

Un **problema de búsqueda** bien formulado (AIMA 4e, cap. 3) se define con cinco componentes:

1. **Estado inicial** `s0`: la situación de partida.

2. **Acciones** `ACTIONS(s)`: el conjunto finito de acciones aplicables en el estado `s`.
3. **Modelo de transición** `RESULT(s, a)`: el estado que resulta de ejecutar la acción `a` en `s`. También llamado función sucesora.
4. **Test de objetivo** `IS-GOAL(s)`: predicado que decide si `s` es un estado meta (puede haber varios).
5. **Costo de acción** `c(s, a, s') >= 0`: el costo de dar ese paso. El costo de un camino es la suma de los costos de sus acciones.

El **espacio de estados** es el grafo dirigido implícito cuyos vértices son todos los estados alcanzables desde `s0` y cuyas aristas son las transiciones. Una **solución** es un camino de `s0` a un estado objetivo; una **solución óptima** es la de costo mínimo.

```
PROBLEMA = (s0, ACTIONS, RESULT, IS-GOAL, c)

función SOLUCION-VALIDA(problema, [a1, ..., an]):
    s ← problema.s0
    costo ← 0
    para cada acción ai:
        si ai ∉ ACTIONS(s): devolver INVÁLIDA
        costo ← costo + c(s, ai, RESULT(s, ai))
        s ← RESULT(s, ai)
    devolver IS-GOAL(s), costo
```

🔍 Estado ≠ nodo

Distinción crucial que la implementación debe respetar:

- Un **estado** es una configuración del mundo (p. ej. la disposición de fichas del 8-puzzle).
- Un **nodo** de búsqueda es una estructura de datos: contiene un estado, un puntero al nodo padre, la acción que lo generó y el costo acumulado `g(n)`. Dos nodos distintos pueden contener el mismo estado (alcanzado por caminos distintos).

Reconstruir la solución consiste en seguir los punteros padre desde el nodo objetivo hasta la raíz.

📐 Abstracción y granularidad

Formular es **abstraer**: decidir qué detalles del mundo entran en el estado. En el problema de viajar de Arad a Bucarest (el mapa de Rumania de AIMA), el estado es solo "ciudad actual"; se descartan clima, combustible y hora. Una abstracción es **válida** si toda solución abstracta puede expandirse a una solución del mundo real, y es **útil** si las acciones abstractas son más fáciles de ejecutar que el problema original. Elegir mal la granularidad es el error de diseño más caro: un estado demasiado fino explota combinatoriamente; uno demasiado grueso pierde soluciones.

📈 Tamaño del espacio y factor de ramificación

Dos números gobiernan la dificultad:

- **Factor de ramificación** `b`: número medio de acciones aplicables por estado.
- **Profundidad** `d`: longitud de la solución más corta.

El árbol de búsqueda tiene O(b^d) nodos. Ejemplos clásicos de tamaño de espacio de estados:

Problema	Estados	Observación
8-puzzle	$9!/2 = 181\,440$	resoluble por fuerza bruta
15-puzzle	$16!/2 \approx 1{,}05 \times 10^{13}$	exige heurísticas
Cubo de Rubik 3×3	$\approx 4{,}3 \times 10^{19}$	resuelto con IDA* + bases de patrones
Ajedrez (posiciones)	$\approx 10^{44}$ (estimación de Shannon)	intratable de forma exacta

Por eso el espacio de estados casi nunca se materializa: se **genera bajo demanda** con `ACTIONS` y `RESULT`.

🧮 Ejemplo trabajado

Formulación completa del **problema de las jarras** (jarra de 4 L y jarra de 3 L, grifo ilimitado; objetivo: exactamente 2 L en la jarra de 4 L):

- **Estado:** par `(x, y)` con `0 ≤ x ≤ 4`, `0 ≤ y ≤ 3`. Espacio: 5 × 4 = 20 estados.
- **Estado inicial:** `(0, 0)`.
- **Acciones:** llenar4, llenar3, vaciar4, vaciar3, verter4→3, verter3→4.
- **Test de objetivo:** `x = 2`.
- **Costo:** 1 por acción (minimizar número de pasos).

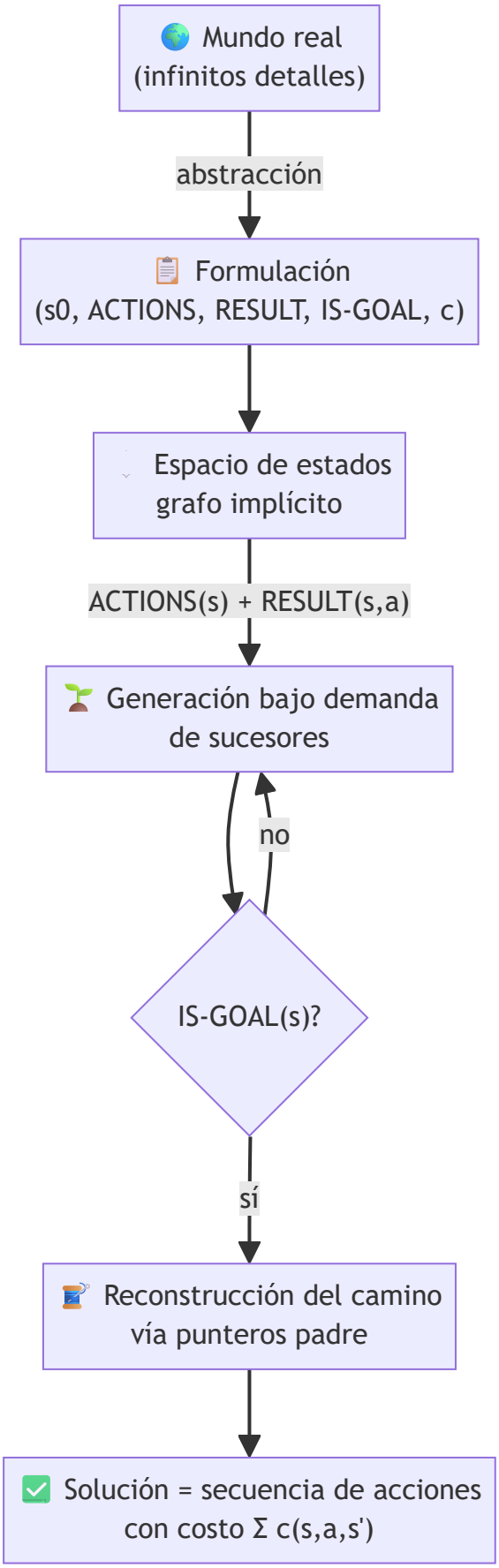
Traza de una solución óptima (6 pasos), aplicando `RESULT` a mano:

```
(0,0) --llenar3-->   (0,3)
(0,3) --verter3→4--> (3,0)
(3,0) --llenar3-->   (3,3)
(3,3) --verter3→4--> (4,2)   # solo cabe 1 L más en la jarra de 4
(4,2) --vaciar4-->   (0,2)
(0,2) --verter3→4--> (2,0)   ✓ IS-GOAL: x = 2
```

Cada línea es verificable aplicando la definición de la acción. Nótese que el estado `(4,2)` codifica todo lo necesario: no hace falta recordar el historial para decidir las acciones siguientes (la formulación es markoviana).

📊 Propiedades y comparación

Criterio de formulación	Estado atómico (esta clase)	Estado factorizado (CSP, clase 018)	Estado estructurado (STRIPS, clase 023)
Representación	caja negra indivisible	vector de variables con dominios	conjunto de literales lógicos
Acciones	enumeradas por función	asignaciones de variables	operadores con precondiciones/efectos
Ventaja	máxima generalidad	propagación de restricciones	acciones descritas sin enumerar estados
Desventaja	el algoritmo no "ve" dentro del estado	solo problemas de asignación	requiere modelado lógico
Algoritmos típicos	BFS, DFS, A*	backtracking, AC-3	GraphPlan, Fast Downward



⚠️ Errores conceptuales frecuentes

1. **"El espacio de estados es una estructura de datos que se construye entera."** Falso: es un grafo *implícito*; se explora generando sucesores bajo demanda. Materializarlo es imposible ya para el 15-puzzle.
2. **Confundir estado con nodo.** El estado describe el mundo; el nodo añade padre, acción y costo acumulado. Ignorar la diferencia impide detectar estados repetidos y reconstruir la solución.
3. **Meter el historial dentro del estado sin necesidad.** Si el objetivo y las acciones solo dependen de la configuración actual, incluir "cómo llegué aquí" multiplica el espacio sin ganar nada.
4. **Suponer que el objetivo es un estado único.** `IS-GOAL` es un predicado: en las jarras, cualquier `(2, y)` sirve; en ajedrez, "jaque mate" describe muchísimas posiciones distintas.
5. **Olvidar el costo y optimizar "número de pasos" por defecto.** Si las acciones tienen costos distintos (peajes, tiempo), la solución con menos pasos puede no ser la óptima; la formulación debe declarar `c` explícitamente.

🚀 Del aprendizaje a la operación

En un sistema real (logística, verificación de software, robótica), la formulación no viene dada: hay que extraerla de requisitos ambiguos y validarla con expertos del dominio. Faltan además: pruebas de que `RESULT` es fiel al sistema real (simulador validado o telemetría), manejo de acciones que fallan o tienen efectos no deterministas (lo que exige MDPs o planificación con contingencias), y monitoreo de que la abstracción sigue siendo válida cuando el entorno cambia. Un modelo de transición equivocado produce planes "óptimos" que fracasan al ejecutarse, y ese error no lo detecta ningún algoritmo de búsqueda.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("search")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4.ª ed.), cap. 3 "Solving Problems by Searching". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Newell, A. y Simon, H. A. (1976). "Computer Science as Empirical Inquiry: Symbols and Search". *Communications of the ACM*, 19(3). https://doi.org/10.1145/360018.360022 — uso: fuente primaria del mecanismo estudiado
- Nilsson, N. J. (1980). *Principles of Artificial Intelligence*. Morgan Kaufmann — caps. 1-2 sobre representación por espacios de estados.
- Stanford Encyclopedia of Philosophy — "Logic and Artificial Intelligence". https://plato.stanford.edu/entries/logic-ai/ — uso: referencia consultada en su fuente original

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P58 · La informática como indagación empírica: símbolos y búsqueda	1976	Enuncia las dos hipótesis que resumen veinte años de IA simbólica: el sistema de símbolos físicos y la búsqueda heurística.	notebook
P64 · Informe sobre un programa general de resolución de problemas	1959	Separa por primera vez el método de resolución del dominio concreto: el análisis medios-fines elige el operador por la diferencia que reduce.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	citada en las referencias de esta clase · caps. 1-2 · obra de referencia de la parte 01
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 3 · obra de referencia de la parte 01

📝 Evaluación completa

❓ Preguntas

1. Define espacios de estados y formulación de problemas sin usar una marca o framework como definición.
2. Explica la relación entre estado, acciones, objetivo, costo.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.

5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

014 — Búsqueda en anchura y profundidad

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `search` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **búsqueda en anchura y profundidad** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar búsqueda en anchura y profundidad usando los conceptos `BFS`, `DFS`, `frontera`, `visitados`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`BFS`, `DFS`, `frontera`, `visitados`

🗺️ Ubicación en el mapa de la IA

BFS y DFS son los dos algoritmos de **búsqueda no informada** fundamentales: exploran el espacio de estados (clase 013) sin ninguna pista sobre dónde está el objetivo. Históricamente son anteriores a la IA misma (teoría de grafos), pero la IA los convirtió en motores de resolución de problemas generales. Son el baseline contra el que se mide toda búsqueda informada (clase 015) y la base de algoritmos posteriores: la profundización iterativa combina ambos, y el backtracking de los CSP (clase 018) es un DFS con poda.

📖 Fundamentos

🌊 BFS: búsqueda en anchura

BFS expande los nodos por **niveles de profundidad**: primero la raíz, luego todos sus sucesores, luego los sucesores de estos. La frontera es una **cola FIFO**. Con costos uniformes, el primer objetivo encontrado está a profundidad mínima.

```
función BFS(problema):
    nodo ← Nodo(problema.s0)
    si IS-GOAL(nodo.estado): devolver nodo
    frontera ← cola FIFO con [nodo]
    alcanzados ← {problema.s0}
    mientras frontera no vacía:
        nodo ← frontera.pop_izquierda()
        para cada hijo en EXPANDIR(problema, nodo):
            s ← hijo.estado
            si IS-GOAL(s): devolver hijo          # test al GENERAR
            si s ∉ alcanzados:
                alcanzados.añadir(s)
                frontera.push_derecha(hijo)
    devolver FALLO
```

Detalle fino (AIMA 4e, §3.4.1): BFS puede aplicar el test de objetivo **al generar** el nodo (early goal test) porque la profundidad garantiza optimalidad en pasos; eso ahorra expandir un nivel entero, O(b^d) nodos.

🕳️ DFS: búsqueda en profundidad

DFS expande siempre el nodo **más profundo** de la frontera (pila LIFO o recursión). Baja por una rama hasta agotar sucesores y entonces **retrocede** (backtracking).

```
función DFS-ARBOL(problema, nodo, límite):
    si IS-GOAL(nodo.estado): devolver nodo
    si límite = 0: devolver CORTE
    para cada hijo en EXPANDIR(problema, nodo):
        resultado ← DFS-ARBOL(problema, hijo, límite - 1)
        si resultado ≠ FALLO: devolver resultado
    devolver FALLO
```

Su virtud no es el tiempo sino la **memoria**: solo guarda la rama actual y los hermanos no expandidos, O(b·m) frente al O(b^d) exponencial de BFS. Su vicio: en grafos con ciclos (o espacios infinitos) sin control de repetidos, **no termina**; y la primera solución encontrada puede ser arbitrariamente peor que la óptima.

🔁 Profundización iterativa (IDS)

IDS ejecuta DFS con límite de profundidad 0, 1, 2, … hasta encontrar solución. Parece derrochador, pero la mayoría de los nodos de un árbol están en el último nivel: el sobrecosto de repetir los niveles superiores es un factor constante (≈ b/(b−1)). IDS combina la memoria O(b·d) de DFS con la completitud y optimalidad (en pasos) de BFS, y es la opción no informada por defecto cuando el espacio es grande y la profundidad de la solución desconocida.

📚 Grafo vs. árbol: el conjunto `alcanzados`

Sin memoria de estados visitados (búsqueda en árbol), los caminos redundantes hacen el trabajo exponencialmente mayor: en una cuadrícula, un árbol de profundidad `d` tiene 4^d hojas pero solo O(d²) celdas distintas. La **búsqueda en grafo** mantiene `alcanzados` y descarta duplicados; cuesta memoria O(|estados|) pero evita la redundancia. La elección árbol/grafo es un trade-off memoria-tiempo, no un detalle de implementación.

🧮 Ejemplo trabajado

Grafo de 6 nodos (aristas no dirigidas), inicio `A`, objetivo `F`:

```
A–B, A–C, B–D, C–E, D–F, E–F
```

Traza BFS (frontera FIFO, `alcanzados` al generar):

Paso	Expande	Genera	Frontera después	Alcanzados
1	A	B, C	[B, C]	{A, B, C}
2	B	D (A ya alcanzado)	[C, D]	{A, B, C, D}
3	C	E	[D, E]	{A, …, E}
4	D	**F** → objetivo al generar	—	—

Solución: `A → B → D → F` (3 aristas, la mínima). Nodos expandidos: 4.

Traza DFS (orden alfabético de sucesores, evitando repetidos en la rama):

```
A → B → D → F ✓   (encuentra F a profundidad 3)
```

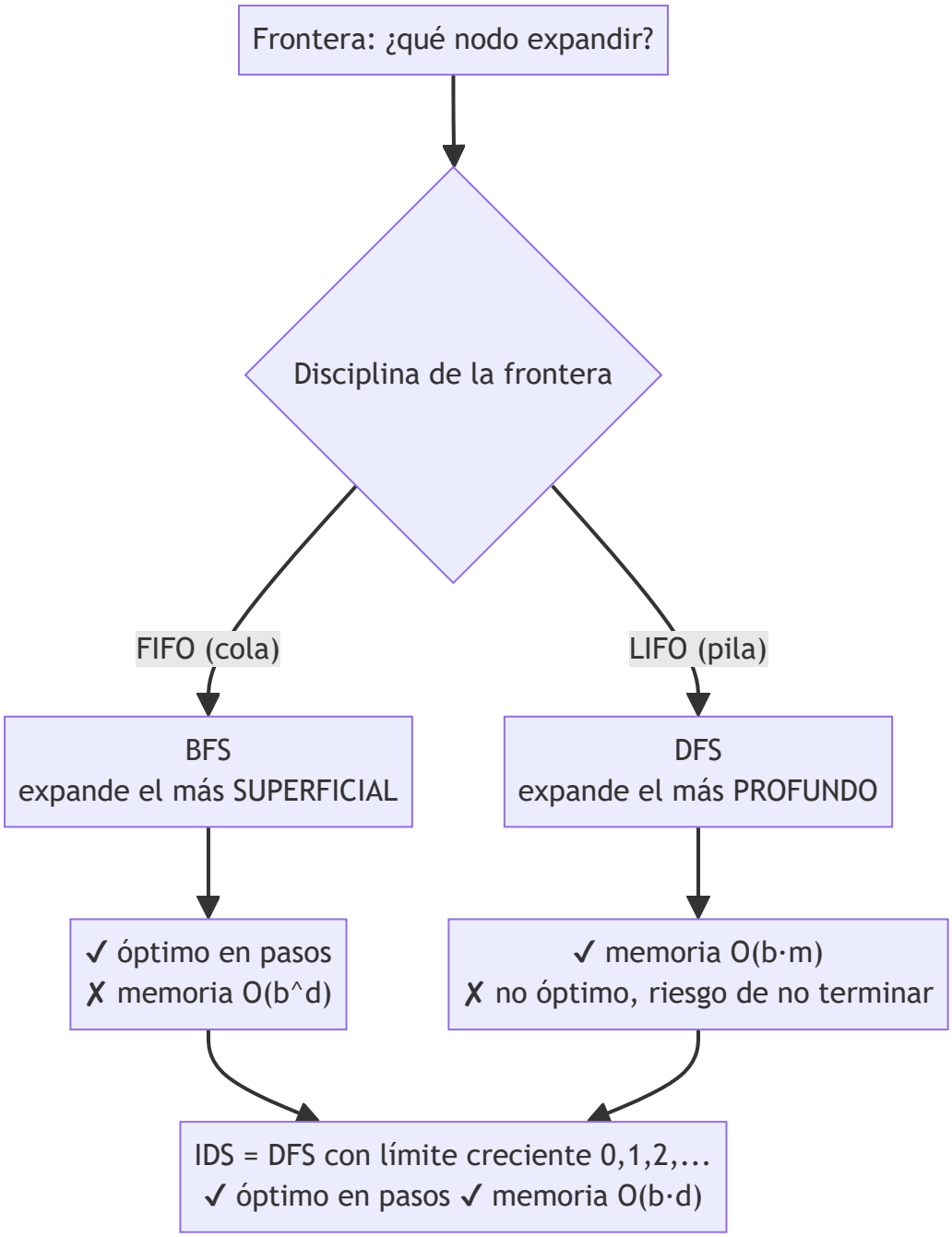
Con este orden DFS tuvo suerte. Si los sucesores de `A` se visitaran en orden `C` primero: `A → C → E → F` también da 3. Pero en el grafo `A–B, B–C, C–F, A–F`, DFS por orden alfabético devuelve `A→B→C→F` (3 aristas) cuando existe `A→F` (1 arista): DFS **no garantiza** el camino más corto; BFS sí.

📊 Propiedades y comparación

Sea `b` el factor de ramificación, `d` la profundidad de la solución más superficial, `m` la profundidad máxima del espacio:

Propiedad	BFS	DFS (árbol)	DFS limitado (ℓ)	IDS
Completo	Sí (b finito)	No (ciclos/∞)	No si ℓ < d	Sí (b finito)
Óptimo (en pasos)	Sí	No	No	Sí
Tiempo	O(b^d)	O(b^m)	O(b^ℓ)	O(b^d)
Memoria	O(b^d)	O(b·m)	O(b·ℓ)	O(b·d)

El cuello de botella práctico de BFS es la **memoria**: con b = 10 y ~1 kB por nodo, la profundidad 10 exige del orden de 10 TB (estimación del propio AIMA); DFS a la misma profundidad usa kilobytes.



⚠️ Errores conceptuales frecuentes

1. **"DFS es más rápido que BFS."** Ambos son O(b^d)/O(b^m) en el peor caso; DFS ahorra *memoria*, no tiempo. Puede ser más rápido o catastróficamente más lento según dónde esté el objetivo.
2. **"BFS siempre encuentra la solución óptima."** Solo en número de pasos (costos uniformes). Con costos de acción distintos, la solución óptima la garantiza costo uniforme/Dijkstra (clase 015), no BFS.
3. **Olvidar `alcanzados` y sorprenderse de que DFS no termina.** En cualquier grafo con ciclos, la búsqueda en árbol pura entra en bucle; hay que controlar repetidos al menos sobre la rama actual.
4. **Creer que IDS "desperdicia" trabajo re-expandiendo niveles.** El último nivel domina el conteo: para b = 10, repetir los niveles superiores añade ~11 % de nodos. Es el precio de tener memoria lineal.

5. **Aplicar el early goal test a algoritmos de costo.** En BFS es válido; en costo uniforme/A *el test debe hacerse al* expandir*, no al generar, o se pierde la optimalidad.

🚀 Del aprendizaje a la operación

Un BFS/DFS de juguete opera sobre un grafo en memoria; en producción (crawlers, análisis de dependencias, verificación de modelos) el grafo vive detrás de APIs con latencia, límites de tasa y fallos parciales, y `alcanzados` puede requerir estructuras aproximadas (filtros de Bloom) o almacenamiento externo cuando hay miles de millones de estados. Además hacen falta cotas de tiempo/memoria con degradación controlada, paralelización de la frontera (BFS por niveles se distribuye bien; DFS no) y métricas de progreso, porque "sigue buscando" y "entró en bucle" son indistinguibles sin instrumentación.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("search")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), §3.3-3.4 "Search Algorithms / Uninformed Search Strategies". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Moore, E. F. (1959). "The shortest path through a maze". *Proceedings of the International Symposium on the Theory of Switching* — origen de BFS para caminos mínimos.
- Korf, R. E. (1985). "Depth-first iterative-deepening: An optimal admissible tree search". *Artificial Intelligence*, 27(1). https://doi.org/10.1016/0004-3702(85)90084-0 — uso: fuente primaria del mecanismo estudiado
- Cormen, T. H. et al. (2022). *Introduction to Algorithms* (4.ª ed.), cap. 20 "Elementary Graph Algorithms" — BFS/DFS con análisis formal.

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P29 · Árbol de pensamientos: resolución deliberada de problemas con modelos de lenguaje grandes	2023	Devuelve la búsqueda clásica al razonamiento: explorar varias ramas, evaluarlas y poder retroceder.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define búsqueda en anchura y profundidad sin usar una marca o framework como definición.
2. Explica la relación entre BFS, DFS, frontera, visitados.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

015 — Costo uniforme, búsqueda voraz y A*

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `search` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **costo uniforme, búsqueda voraz y a*** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar costo uniforme, búsqueda voraz y a* usando los conceptos `UCS`, `greedy`, `A*`, `heurística`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`UCS`, `greedy`, `A*`, `heurística`

🗺️ Ubicación en el mapa de la IA

Esta clase da el salto de la búsqueda ciega (clase 014) a la **búsqueda informada**: usar conocimiento del dominio, empaquetado en una función heurística, para expandir menos nodos. A *(Hart, Nilsson y Raphael, 1968, desarrollado para el robot Shakey) es probablemente el algoritmo más influyente de la IA clásica: sigue siendo el estándar en videojuegos, robótica y planificación (los planificadores tipo Fast Downward son A* sobre espacios STRIPS). Prepara directamente la clase 016 (qué hace buena a una heurística).

📖 Fundamentos

🧭 Un solo algoritmo, tres funciones de evaluación

Los tres métodos son **best-first search** con frontera de prioridad ordenada por una función `f(n)`; solo cambia `f`:

Algoritmo	f(n)	Usa costo real g	Usa heurística h
Costo uniforme (UCS / Dijkstra)	g(n)	✓	✗
Voraz (greedy best-first)	h(n)	✗	✓
A*	g(n) + h(n)	✓	✓

donde `g(n)` es el costo acumulado desde el inicio hasta `n`, y `h(n)` es una **estimación** del costo desde `n` hasta el objetivo, con `h(objetivo) = 0`.

```
función BEST-FIRST(problema, f):
    nodo ← Nodo(problema.s0, g=0)
    frontera ← cola de prioridad ordenada por f, con [nodo]
    alcanzados ← {problema.s0: nodo}
    mientras frontera no vacía:
        nodo ← frontera.pop_min()
        si IS-GOAL(nodo.estado): devolver nodo    # test al EXPANDIR
        para cada hijo en EXPANDIR(problema, nodo):
            s ← hijo.estado
            si s ∉ alcanzados o hijo.g < alcanzados[s].g:
                alcanzados[s] ← hijo              # re-apertura por camino mejor
                frontera.push(hijo)
    devolver FALLO
```

Dos detalles no negociables: el test de objetivo se hace **al expandir** (al sacar de la cola), no al generar — de lo contrario UCS y A *pierden la optimalidad —; y un estado ya alcanzado se reabre** si aparece un camino más barato.

💰 Costo uniforme: optimalidad sin información

UCS expande siempre el nodo de menor `g`. Cuando extrae un nodo de la frontera, ya no puede existir camino más barato hasta él (todos los costos son ≥ 0): por eso es **óptimo y completo** si los costos de acción son ≥ ε > 0. Es la generalización de BFS a costos arbitrarios y equivale al algoritmo de Dijkstra con un solo destino. Su complejidad se expresa en función del costo óptimo `C*`: O(b^(1+⌊C*/ε⌋)), que puede ser mucho peor que O(b^d) si hay acciones muy baratas.

🎯 Voraz: rápida y ciega al pasado

La búsqueda voraz expande el nodo que *parece* más cercano al objetivo (`f = h`). Suele llegar rápido, pero ignora lo gastado: puede tomar caminos largos que "apuntan bien" y, con búsqueda en árbol, entrar en bucles. No es óptima ni completa (en árbol); su interés es que expande muy pocos nodos cuando la heurística es razonable.

⭐ A*: lo mejor de ambos

A *ordena por `f(n) = g(n) + h(n)`: el costo estimado de la mejor solución* que pasa por n*. Sus garantías dependen de la heurística (definiciones formales en la clase 016):

- Con `h` **admisible** (nunca sobreestima): A *con búsqueda en árbol es óptimo**.
- Con `h` **consistente** (`h(n) ≤ c(n, a, n') + h(n')`): A* con búsqueda en grafo es óptimo y nunca necesita reabrir nodos; los valores `f` a lo largo de cualquier camino son no decrecientes.

- A *es óptimamente eficiente**: ningún algoritmo óptimo que use la misma `h` puede evitar expandir los nodos con `f(n) < C*`.

Su límite práctico es la memoria: guarda toda la frontera y los alcanzados, exponencial en el peor caso. Variantes como IDA *y SMA* cambian tiempo por memoria.

🧮 Ejemplo trabajado

Grafo dirigido con costos; inicio `S`, objetivo `G`. Heurística entre paréntesis:

```
S(6) --2--> A(4)     S --3--> B(2)
A --3--> C(1)        B --4--> C(1)
C --2--> G(0)        B --6--> G(0)
```

Verificación de admisibilidad: los costos reales al objetivo son h*(S)=7,* h(A)=5, h*(B)=6,* h(C)=2; cada `h` está por debajo ✓.

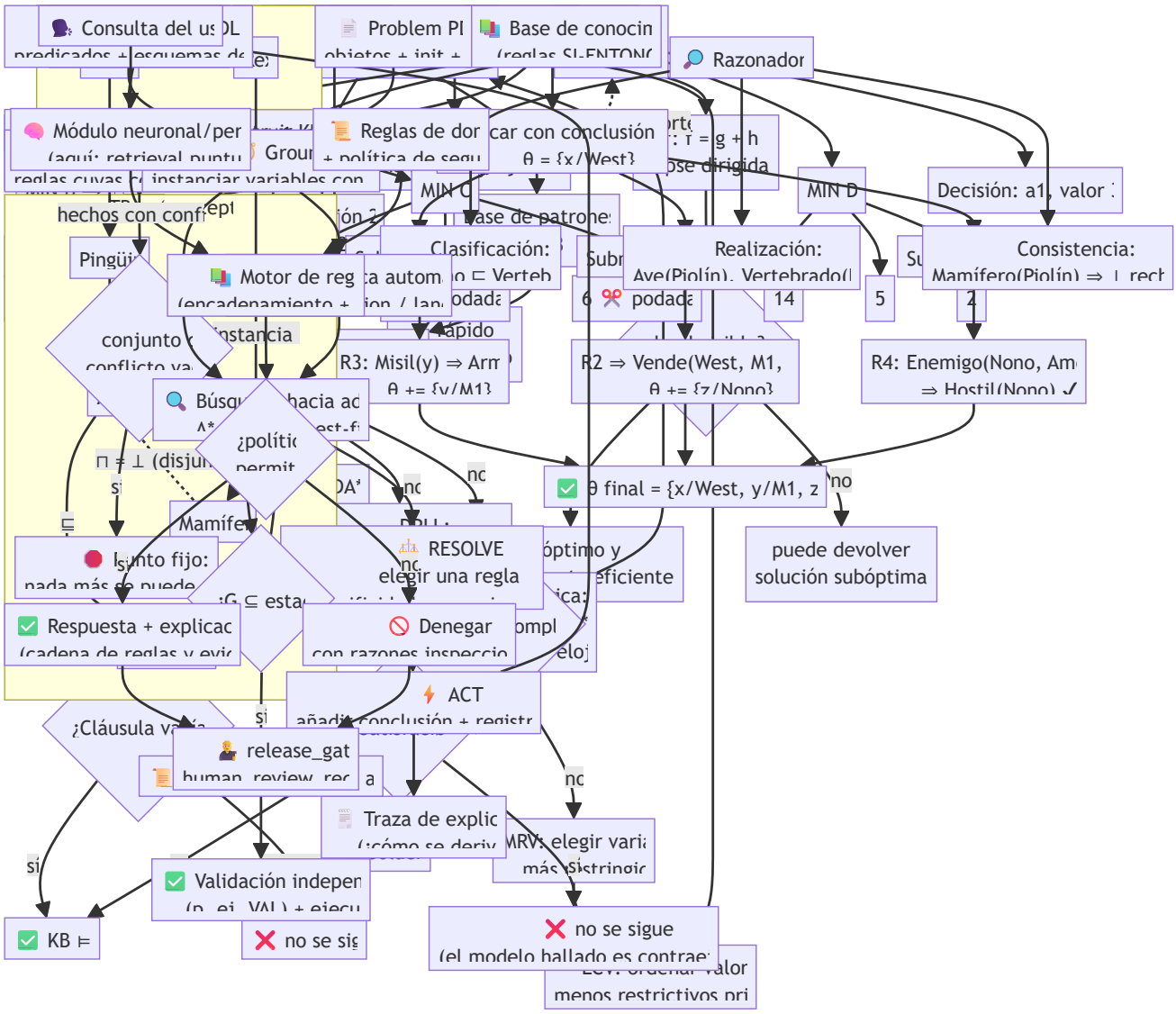
Traza de A* (`f = g + h`; la frontera muestra `estado: g+h=f`):

Paso	Expande	Frontera después
0	—	S: 0+6=6
1	S (f=6)	A: 2+4=6, B: 3+2=5
2	B (f=5)	A: 6, C: 7+1=8 (vía B), G: 9+0=9
3	A (f=6)	C: **5+1=6** (vía A, reabre: 5 < 7), G: 9
4	C (f=6)	G: 7+0=7 (vía A-C, mejora 9)
5	G (f=7)	✓ solución S→A→C→G, costo 7

- **Voraz** habría hecho: S → B (h=2) → G (h=0): devuelve S→B→G con costo 9 ≠ óptimo.
- **UCS** expande S(0), A(2), B(3), C(5), G(7): mismo resultado que A *pero ordenando por* `g` *puro; en grafos grandes expande muchos más nodos que A*.
- El paso 3 muestra la **re-apertura**: C se había alcanzado vía B con g=7 y se sustituye al hallar g=5. Esta `h` es admisible pero **no consistente**: h(S)=6 > c(S,B)+h(B)=3+2=5, y esa subestimación relativa de B hizo que B se expandiera "demasiado pronto"; por eso hubo reapertura.

📊 Propiedades y comparación

Propiedad	UCS	Voraz	A* (h admisible/consistente)
Completo	Sí (costos ≥ ε > 0)	Solo en grafo finito	Sí
Óptimo	Sí	No	Sí
Tiempo	O(b^(1+⌊C*/ε⌋))	O(b^m), a menudo mucho menos	exponencial en el error de h
Memoria	como el tiempo	como el tiempo	guarda frontera + alcanzados
Necesita h	No	Sí	Sí
Nodos expandidos	muchos (círculo de radio C*)	pocos	intermedio: todos con f(n) < C*



⚠️ Errores conceptuales frecuentes

1. **Testear el objetivo al generar el nodo (como en BFS).** UCS y A *deben testearlo al* expandir*; el primer camino que genera el objetivo puede no ser el más barato (véase el paso 2 vs. 5 del ejemplo).
2. **"A* siempre es más rápido que UCS."** Solo si `h` aporta información; con `h = 0`, A es *UCS. Y una* `h` *cara de calcular puede hacer que A* pierda en tiempo de reloj aunque expanda menos nodos.
3. **"Greedy no sirve porque no es óptimo."** En problemas donde cualquier solución razonable vale y el tiempo apremia, voraz (o weighted A*, `f = g + w·h`) es la elección correcta consciente del trade-off.
4. **Ignorar la re-apertura de nodos.** Con heurística admisible pero no consistente, marcar estados como "cerrados para siempre" rompe la optimalidad de A* en grafo.
5. **Confundir el costo del camino con la profundidad.** UCS con costos uniformes degenera en BFS; la diferencia solo aparece cuando `c` varía por acción.

🚀 Del aprendizaje a la operación

Entre este laboratorio y un A *de producción (navegación, videojuegos, ruteo logístico) median: heurísticas específicas del dominio validadas empíricamente (clase 016), estructuras de datos afinadas (montículos con decrease-key o cubetas), preprocesamiento del grafo (contraction hierarchies en mapas reales, que dejan a Dijkstra puro obsoleto a escala continental), replanificación incremental cuando el mundo cambia (D* Lite en robótica) y límites de memoria explícitos con degradación a variantes anytime. Además, el grafo real viene de datos con errores: un costo mal cargado invalida la "optimalidad" sin que el algoritmo lo detecte.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("search")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Hart, P. E., Nilsson, N. J. y Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths". *IEEE Trans. on Systems Science and Cybernetics*, 4(2). https://doi.org/10.1109/TSSC.1968.300136 — uso: fuente primaria del mecanismo estudiado
- Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs". *Numerische Mathematik*, 1. https://doi.org/10.1007/BF01386390 — uso: fuente primaria del mecanismo estudiado
- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), §3.5 "Informed (Heuristic) Search Strategies". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley — análisis formal de A* y sus variantes.

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P67 · Una base formal para la determinación heurística de caminos de coste mínimo	1968	Convierte la heurística de recurso práctico en garantía demostrable: si nunca sobrestima, el camino encontrado es óptimo.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Pearl, J. — *Heuristics: Intelligent Search Strategies for Computer Problem Solving*	1984	*sin localizador verificado*	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define costo uniforme, búsqueda voraz y a* sin usar una marca o framework como definición.
2. Explica la relación entre UCS, greedy, A*, heurística.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

016 — Diseño y validación de heurísticas

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `search` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **diseño y validación de heurísticas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar diseño y validación de heurísticas usando los conceptos `admisibilidad`, `consistencia`, `dominancia`, `costo`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`admisibilidad`, `consistencia`, `dominancia`, `costo`

🗺️ Ubicación en el mapa de la IA

Las garantías de A *(clase 015) valen lo que valga su heurística: esta clase estudia qué hace a una heurística* correcta *(admisibilidad, consistencia) y* buena *(dominancia, factor de ramificación efectivo). La idea central — derivar heurísticas resolviendo problemas relajados** — reaparece en toda la IA: las heurísticas de los planificadores PDDL (clase 023) son relajaciones automáticas, y las bases de datos de patrones anticipan la idea de precomputar conocimiento que luego guía la búsqueda, como hoy hacen las funciones de valor aprendidas en RL.

📖 Fundamentos

✅ Admisibilidad

Una heurística `h` es **admisible** si nunca sobreestima el costo real mínimo al objetivo:

```
∀n:  0 ≤ h(n) ≤ h*(n)        donde h*(n) = costo óptimo real de n al objetivo
```

Es una garantía de *optimismo*: la solución puede ser peor de lo que `h` promete, nunca mejor. Con `h` admisible, A *en árbol devuelve siempre una solución óptima: si devolviera una subóptima de costo C > C*, algún nodo del camino óptimo tendría f(n) = g(n) + h(n) ≤ C* < C y habría sido expandido antes.

🔗 Consistencia (monotonía)

`h` es **consistente** si cumple la desigualdad triangular con cada arista:

```
∀n, a, n' sucesor:  h(n) ≤ c(n, a, n') + h(n')      y  h(objetivo) = 0
```

Consistencia ⇒ admisibilidad (se prueba por inducción sobre la longitud del camino al objetivo), pero no al revés. Su consecuencia operativa: los valores `f` son no decrecientes a lo largo de cualquier camino, así que la primera vez que A *en grafo* expande *un estado ya lo hace con su `g` óptimo y nunca hay que reabrir nodos**. Casi todas las heurísticas naturales (distancias geométricas, relajaciones) son consistentes; construir una admisible-pero-inconsistente requiere cierto esfuerzo deliberado.

🏆 Dominancia y calidad

Si `h2(n) ≥ h1(n)` para todo `n` (ambas admisibles), `h2` **domina** a `h1` y A *con `h2` nunca expande más nodos que con `h1` (salvo empates en f = C)*. Regla práctica: entre heurísticas admisibles, gana la más grande. De hecho, el **máximo** de varias admisibles es admisible y las domina a todas:

```
h(n) = max(h1(n), h2(n), ..., hk(n))
```

La calidad se mide empíricamente con el **factor de ramificación efectivo** b*: si A* expandió N nodos para una solución a profundidad d, b *es la solución de `N + 1 = 1 + b* + (b*)² + ... + (b*)^d`. Una heurística buena acerca b* a 1. Para el 8-puzzle a profundidad d = 12 (datos de AIMA): IDS expande 3 644 035 nodos (b *≈ 2,78), A* con fichas mal colocadas 227 (b *≈ 1,42), A* con Manhattan 73 (b* ≈ 1,24).

🛠️ De dónde salen: problemas relajados

Método sistemático: **eliminar restricciones** del problema. El costo óptimo del problema relajado es una cota inferior del original (todo camino del original sigue siendo válido en el relajado), luego es admisible; y como se calcula sobre el problema relajado *resuelto exactamente*, suele ser consistente.

- 8-puzzle, regla original: "una ficha se mueve a la casilla adyacente vacía".
- Relajación 1: "una ficha se mueve a cualquier casilla" → `h1` = número de fichas mal colocadas.
- Relajación 2: "una ficha se mueve a una casilla adyacente (aunque esté ocupada)" → `h2` = suma de distancias Manhattan. `h2` domina a `h1`.
- Rutas en mapa: relajar "moverse por carreteras" a "volar en línea recta" → distancia euclídea.

Otras dos fuentes: **bases de datos de patrones** (resolver exhaustivamente subproblemas — p. ej. solo las fichas 1-4 — y tabular los costos exactos, Culberson y Schaeffer 1998) y **aprendizaje** de h a partir de instancias resueltas (sin garantía de admisibilidad, salvo que se acote).

⚖️ El trade-off real

Una heurística más informada expande menos nodos pero cuesta más por nodo. El tiempo total es ≈ `nodos_expandidos × (costo_generación + costo_h)`. Una `h` perfecta (h = h*) *reduce la búsqueda a seguir el camino óptimo, pero calcular h* equivale a resolver el problema. El punto óptimo está en heurísticas baratas y razonablemente informadas — o precomputadas, pagando memoria en vez de tiempo.

🧮 Ejemplo trabajado

Estado del 8-puzzle (0 = hueco) y objetivo estándar:

```
estado s:  7 2 4        objetivo:  0 1 2
           5 0 6                   3 4 5
           8 3 1                   6 7 8
```

h1 (fichas mal colocadas): comparando casilla a casilla, las 8 fichas están fuera de su lugar → `h1(s) = 8`.

h2 (Manhattan): distancia |Δfila| + |Δcolumna| de cada ficha a su posición objetivo:

Ficha	1	2	3	4	5	6	7	8
Posición actual (f,c)	(2,2)	(0,1)	(2,1)	(0,2)	(1,0)	(1,2)	(0,0)	(2,0)
Posición objetivo (f,c)	(0,1)	(0,2)	(1,0)	(1,1)	(1,2)	(2,0)	(2,1)	(2,2)
Distancia	3	1	2	1	2	3	2	2

`h2(s) = 3+1+2+1+2+3+2+2 = 16`. El costo real es `h*(s) = 26` (instancia usada en AIMA): ambas son cotas inferiores (8 ≤ 16 ≤ 26 ✓) y `h2` domina a `h1`, así que A* *con* `h2` *expandirá menos nodos. Nótese cuánta información pierde* `h1`*: sabe* que *las fichas están mal, no* cuán lejos*.

📊 Propiedades y comparación

Heurística (8-puzzle)	Admisible	Consistente	Informatividad	Costo de cálculo
h0 = 0 (UCS)	Sí	Sí	nula	O(1)
h1 = fichas mal colocadas	Sí	Sí	baja	O(k)
h2 = Manhattan	Sí	Sí	media (domina h1)	O(k)
Conflictos lineales + Manhattan	Sí	Sí	alta	$O(k^2)$
Base de datos de patrones	Sí	Sí	muy alta	O(1) consulta + memoria/precómputo
h aprendida sin cota	No garantizado	No garantizado	variable	según modelo

⚠️ Errores conceptuales frecuentes

1. **"Si h es admisible, A* es rápido."** Admisibilidad garantiza *optimalidad del resultado*, no eficiencia: h = 0 es admisible y da UCS. La velocidad depende de cuán cerca esté h de h*.
2. **Confundir admisible con consistente.** Toda consistente es admisible; lo inverso es falso. Con admisible-no-consistente, A* en grafo debe permitir reabrir nodos o pierde optimalidad.
3. **Sumar heurísticas admisibles y asumir que la suma es admisible.** En general sobreestima (cuenta los mismos movimientos dos veces). Solo es válido si son **aditivas/disjuntas** (p. ej. bases de patrones disjuntas, donde cada movimiento se acredita a un solo patrón). La combinación siempre segura es `max`.
4. **Escalar la heurística "para que busque más rápido" y esperar optimalidad.** `w·h` con $w > 1$ (weighted A*) puede sobreestimar: gana velocidad y pierde la garantía, acotando el resultado por $w \cdot C$.* Es un trade-off legítimo, pero hay que declararlo.
5. **Validar solo con una instancia.** La calidad de una heurística se reporta sobre un conjunto de instancias (media de nodos expandidos, b*, tiempo total), no sobre el caso donde funcionó bien.

🚀 Del aprendizaje a la operación

En producción, la heurística se elige con *benchmarks* del dominio real, no con intuición: hay que medir nodos expandidos, tiempo de reloj y memoria sobre cargas representativas, y repetir la medición cuando el dominio cambia (un mapa con obras rompe la calibración de una h aprendida). Las bases de patrones exigen decidir cuánta memoria dedicar al precómputo y regenerarla en cada cambio de dominio. Y si se usan heurísticas aprendidas sin garantía de admisibilidad, el sistema debe declarar que las soluciones pueden ser subóptimas y acotar empíricamente cuánto.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("search")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;

- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), §3.6 "Heuristic Functions". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Pearl, J. (1984). *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley — el tratado clásico sobre heurísticas admisibles y su análisis.
- Culberson, J. C. y Schaeffer, J. (1998). "Pattern Databases". *Computational Intelligence*, 14(3). https://doi.org/10.1111/0824-7935.00065 — uso: fuente primaria del mecanismo estudiado
- Felner, A., Korf, R. E. y Hanan, S. (2004). "Additive Pattern Database Heuristics". *JAIR*, 22. https://doi.org/10.1613/jair.1480 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P29 · Árbol de pensamientos: resolución deliberada de problemas con modelos de lenguaje grandes	2023	Devuelve la búsqueda clásica al razonamiento: explorar varias ramas, evaluarlas y poder retroceder.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Pearl, J. — *Heuristics: Intelligent Search Strategies for Computer Problem Solving*	1984	*sin localizador verificado*	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define diseño y validación de heurísticas sin usar una marca o framework como definición.
2. Explica la relación entre admisibilidad, consistencia, dominancia, costo.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

017 — Juegos: minimax y poda alfa-beta

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `search` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **juegos: minimax y poda alfa-beta** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar juegos: minimax y poda alfa-beta usando los conceptos `minimax`, `alfa-beta`, `adversarial`, `utilidad`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`minimax`, `alfa-beta`, `adversarial`, `utilidad`

🗺️ Ubicación en el mapa de la IA

La búsqueda adversarial extiende la búsqueda en espacios de estados (clases 013-016) a entornos con un **oponente** que elige contra nosotros. Minimax formaliza la idea de von Neumann (teorema minimax, 1928) como algoritmo; el programa de damas de Samuel (1959) y Deep Blue (1997) son sus hitos. La línea evolutiva continúa: AlphaGo (2016) sustituyó la función de evaluación manual por redes neuronales y el barrido exhaustivo por búsqueda de árbol Monte Carlo, pero el esqueleto — explorar un árbol de jugadas alternadas y respaldar valores — sigue siendo el de esta clase.

📖 Fundamentos

🎲 Formulación de un juego

Un juego de suma cero, dos jugadores, turnos alternos e información perfecta se define con: estado inicial `s0`, `TO-MOVE(s)` (a quién le toca), `ACTIONS(s)`, `RESULT(s, a)`, `IS-TERMINAL(s)` y `UTILITY(s, jugador)` (valor numérico del estado terminal: p. ej. +1 victoria, 0 tablas, −1 derrota). **Suma cero** significa que lo que gana MAX lo pierde MIN, así que basta un solo número por estado.

♟️ Minimax

El **valor minimax** de un estado es la utilidad que obtiene MAX si ambos juegan perfectamente de ahí en adelante:

```
MINIMAX(s) =
    UTILITY(s, MAX)                                 si IS-TERMINAL(s)
    max_{a ∈ ACTIONS(s)} MINIMAX(RESULT(s, a))      si TO-MOVE(s) = MAX
    min_{a ∈ ACTIONS(s)} MINIMAX(RESULT(s, a))      si TO-MOVE(s) = MIN
```

El algoritmo es un recorrido DFS del árbol de juego que **respalda** (backs up) los valores desde las hojas: cada nodo MAX toma el máximo de sus hijos, cada nodo MIN el mínimo. Es óptimo *contra un oponente óptimo*; contra un oponente que falla, garantiza al menos ese valor (puede existir otra línea que explote mejor los errores, pero minimax nunca hace peor que su garantía). Complejidad: tiempo O(b^m), memoria O(b·m) — inviable para ajedrez (b ≈ 35, m ≈ 80).

🎀 Poda alfa-beta

Idea: mantener durante el recorrido dos cotas del valor que los jugadores ya tienen garantizado en el camino desde la raíz:

- **α**: la mejor opción (máxima) asegurada para MAX hasta ahora.
- **β**: la mejor opción (mínima) asegurada para MIN hasta ahora.

Si en un nodo MIN el valor cae a `v ≤ α`, MAX nunca permitirá llegar aquí (ya tiene algo mejor): se **poda** el resto de hijos. Simétricamente en nodos MAX cuando `v ≥ β`.

```
función ALFA-BETA(s, α, β):
    si IS-TERMINAL(s): devolver UTILITY(s, MAX)
    si TO-MOVE(s) = MAX:
        v ← −∞
        para cada a en ACTIONS(s):
            v ← max(v, ALFA-BETA(RESULT(s,a), α, β))
            si v ≥ β: devolver v          # poda: MIN nunca dejará llegar aquí
            α ← max(α, v)
        devolver v
    si no:  # MIN
        v ← +∞
        para cada a en ACTIONS(s):
            v ← min(v, ALFA-BETA(RESULT(s,a), α, β))
            si v ≤ α: devolver v          # poda: MAX tiene algo mejor
            β ← min(β, v)
        devolver v
```

La poda **no altera el resultado**: devuelve exactamente el valor minimax. Con **ordenación perfecta** de jugadas (probar primero la mejor) examina O(b^(m/2)) nodos — duplica la profundidad alcanzable con el mismo presupuesto (Knuth y Moore, 1975); con orden aleatorio, ≈ O(b^(3m/4)). Por eso los motores invierten tanto en ordenar jugadas (jugada de la tabla de transposición primero, capturas, killer moves).

⏱️ Búsqueda con recursos finitos

Ningún juego interesante se explora hasta las hojas. Se corta a profundidad `d` y se aplica una **función de evaluación** `EVAL(s)`: una estimación de la utilidad esperada, típicamente lineal en rasgos (`EVAL(s) = Σ wi·fi(s)`; en ajedrez: material, movilidad, estructura de peones). Problemas asociados: el **efecto horizonte** (una pérdida inevitable se "empuja" más allá del corte con jadeos tácticos) se mitiga con **búsqueda de quiescencia** (extender el corte hasta posiciones tranquilas, sin capturas pendientes); las **tablas de transposición** cachean valores de estados repetidos alcanzados por distinto orden de jugadas; la **profundización iterativa** da control de tiempo y mejora la ordenación con la mejor jugada de la iteración anterior.

🧮 Ejemplo trabajado

Árbol de profundidad 2: la raíz es MAX con tres jugadas (a1, a2, a3); cada hijo es un nodo MIN con tres hojas:

```
              MAX
       a1     /  a2 |  a3 \
       MIN B     MIN C     MIN D
      /3 12 8\  /2 4 6\   /14 5 2\
```

Minimax puro: B = min(3,12,8) = 3; C = min(2,4,6) = 2; D = min(14,5,2) = 2; raíz = max(3,2,2) = **3** → jugar a1. Hojas evaluadas: 9.

Alfa-beta (izquierda a derecha, raíz con α=−∞, β=+∞):

```
B: ve 3, 12, 8 → B = 3.  Raíz: α = 3.
C: primera hoja 2 → v = 2 ≤ α = 3 → PODA (no mira 4 ni 6). C ≤ 2, irrelevante.
D: primera hoja 14 → v = 14, β_D = 14... segunda hoja 5 → v = 5,
   tercera hoja 2 → v = 2 ≤ α = 3 → D ≤ 2, descartado.
Raíz = 3, jugar a1. Hojas evaluadas: 7 de 9.
```

Con ordenación óptima (explorar D en orden 2, 5, 14) la poda habría sido inmediata tras la primera hoja de C y de D: 5 hojas. La ganancia de alfa-beta *depende del orden*, no de la suerte del árbol.

📊 Propiedades y comparación

Propiedad	Minimax	Alfa-beta	Alfa-beta + orden perfecto	MCTS (contraste)
Resultado	valor exacto	idéntico a minimax	idéntico	estimación estadística
Tiempo	O(b^m)	≈ O(b^(3m/4)) típico	O(b^(m/2))	presupuesto fijo de simulaciones
Memoria	O(b·m)	O(b·m)	O(b·m)	árbol parcial en memoria
Requiere EVAL	sí (con corte)	sí (con corte)	sí	no (rollouts) o red de valor
Dónde brilla	árboles pequeños	juegos tácticos (ajedrez)	con buenas jugadas-candidato	b enorme, EVAL difícil (Go)

⚠️ Errores conceptuales frecuentes

1. **"Alfa-beta es una aproximación que sacrifica exactitud por velocidad."** Falso: devuelve exactamente el valor minimax. Lo aproximado es la función de evaluación al cortar profundidad, no la poda.
2. **"Minimax asume que el rival es perfecto, así que contra malos jugadores es malo."** Contra un rival subóptimo minimax obtiene *al menos* su valor garantizado. Lo que sí es cierto: no modela al oponente, así que no *explota* sus debilidades.
3. **Podar comparando valores de hojas en vez de cotas del camino.** α y β son garantías acumuladas *desde la raíz*; usarlas como valores locales del nodo produce podas incorrectas.
4. **Ignorar el orden de exploración.** Alfa-beta con mal orden se acerca a minimax puro. La mitad de la ingeniería de un motor real es ordenación de jugadas.
5. **Evaluar en posiciones "calientes".** Cortar en medio de un intercambio de capturas hace que EVAL mida ruido (efecto horizonte); la quiescencia existe precisamente para eso.

🚀 Del aprendizaje a la operación

Entre este laboratorio y un motor de juego real median: función de evaluación calibrada (hoy, redes NNUE entrenadas con millones de posiciones), tablas de transposición con gestión de memoria y colisiones, control de tiempo por jugada con profundización iterativa, paralelización (Lazy SMP) y bancos de pruebas de regresión (miles de partidas contra versiones anteriores para validar cada cambio con significancia estadística). Además, para juegos con azar o información oculta (backgammon, póker) minimax puro no basta: hacen falta expectiminimax o equilibrios de teoría de juegos.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("search")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;

- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), cap. 5 "Adversarial Search and Games". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Knuth, D. E. y Moore, R. W. (1975). "An Analysis of Alpha-Beta Pruning". *Artificial Intelligence*, 6(4). https://doi.org/10.1016/0004-3702(75)90019-3 — uso: fuente primaria del mecanismo estudiado
- Shannon, C. E. (1950). "Programming a Computer for Playing Chess". *Philosophical Magazine*, 41(314) — el paper fundacional del ajedrez computacional.
- Campbell, M., Hoane, A. J. y Hsu, F. (2002). "Deep Blue". *Artificial Intelligence*, 134(1-2). https://doi.org/10.1016/S0004-3702(01)00129-1 — uso: fuente primaria del mecanismo estudiado
- Silver, D. et al. (2016). "Mastering the game of Go with deep neural networks and tree search". *Nature*, 529. https://doi.org/10.1038/nature16961 — uso: fuente primaria del mecanismo estudiado

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P27 · Dominar el go con redes neuronales profundas y búsqueda en árbol	2016	Une las dos tradiciones de la IA: la búsqueda simbólica de la parte 01 y el aprendizaje profundo de la parte 04, en un solo sistema.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 5 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define juegos: minimax y poda alfa-beta sin usar una marca o framework como definición.
2. Explica la relación entre minimax, alfa-beta, adversarial, utilidad.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

018 — Problemas de satisfacción de restricciones

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **problemas de satisfacción de restricciones** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar problemas de satisfacción de restricciones usando los conceptos `CSP`, `backtracking`, `consistencia`, `restricciones`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`CSP`, `backtracking`, `consistencia`, `restricciones`

🗺️ Ubicación en el mapa de la IA

Los CSP abandonan el estado atómico de las clases 013-015 por una **representación factorizada**: el estado es un conjunto de variables con dominios, y esa estructura interna permite algo imposible en búsqueda ciega — *deducir* qué ramas son inútiles antes de explorarlas (propagación de restricciones). Es el puente entre búsqueda y lógica: un CSP booleano es exactamente el problema SAT de la clase 019. Sus herederos industriales (programación con restricciones, solvers SMT como Z3) resuelven hoy horarios, verificación de hardware y configuración de productos.

📖 Fundamentos

🧩 Definición

Un **CSP** es una terna `(X, D, C)`:

- **Variables** `X = {X1, ..., Xn}`.
- **Dominios** `D = {D1, ..., Dn}`: valores posibles de cada variable.

- **Restricciones** `C`: cada una es un par `⟨alcance, relación⟩`, p. ej. `⟨(X1, X2), X1 ≠ X2⟩`. Las más comunes son **binarias** (dos variables); todo CSP puede binarizarse.

Una **asignación** es consistente si no viola restricciones, y completa si cubre todas las variables. Una **solución** = completa y consistente. Decidir si existe solución es NP-completo en general (3-colorabilidad de grafos es un CSP). El **grafo de restricciones** (vértice por variable, arista por restricción binaria) revela estructura explotable: los CSP con grafo en árbol se resuelven en $O(n \cdot d^2)$.

⬅ Búsqueda con backtracking

El algoritmo base asigna variables una a una en DFS y retrocede al detectar una violación:

```
función BACKTRACK(csp, asignación):
    si asignación completa: devolver asignación
    var ← ELEGIR-VARIABLE-NO-ASIGNADA(csp, asignación)     # heurística MRV
    para cada valor en ORDENAR-VALORES(var, csp, asignación):  # LCV
        si valor consistente con asignación:
            asignación[var] ← valor
            inferencias ← INFERIR(csp, var, valor)          # forward checking / MAC
            si inferencias ≠ fallo:
                resultado ← BACKTRACK(csp, asignación)
                si resultado ≠ fallo: devolver resultado
            deshacer asignación e inferencias
    devolver fallo
```

Aprovecha la **conmutatividad**: el orden de asignación no cambia el conjunto de soluciones, así que basta ramificar sobre *una* variable por nivel (b = d por nivel, no n·d). Las heurísticas generales que lo hacen práctico:

- **MRV** (minimum remaining values): elegir la variable con menos valores legales — "fail first", detecta callejones pronto.
- **Grado**: como desempate, la variable con más restricciones sobre variables libres.
- **LCV** (least constraining value): probar primero el valor que menos recorta los dominios vecinos — "fail last" en los valores, porque solo necesitamos una solución.

📡 Propagación: forward checking y AC-3

Forward checking: al asignar `X = v`, eliminar de los dominios de las variables vecinas los valores incompatibles con `v`. Detecta el fallo cuando un dominio queda vacío, pero no propaga en cadena.

Consistencia de arco: el arco `(Xi, Xj)` es consistente si para todo valor de `Di` existe algún valor de `Dj` que satisface la restricción. **AC-3** (Mackworth, 1977) impone consistencia de arco en todo el CSP:

```
función AC-3(csp):
    cola ← todos los arcos (Xi, Xj) del csp
    mientras cola no vacía:
        (Xi, Xj) ← cola.pop()
        si REVISAR(Xi, Xj):                 # ¿se recortó Di?
            si Di vacío: devolver falso     # inconsistencia detectada
            para cada Xk vecino de Xi, k ≠ j:
                cola.añadir((Xk, Xi))       # re-examinar en cadena

función REVISAR(Xi, Xj):
    recortado ← falso
    para cada x en Di:
        si ningún y en Dj satisface la restricción(x, y):
            eliminar x de Di; recortado ← verdadero
    devolver recortado
```

Complejidad O(c·d³) con `c` arcos y dominios de tamaño `d`. AC-3 puede usarse como preproceso o **dentro** de la búsqueda tras cada asignación (algoritmo MAC, *maintaining arc consistency*), el estándar en solvers serios. Importante: consistencia de arco ≠ solución; puede quedar un CSP arco-consistente sin solución (la detección completa exige consistencias superiores o búsqueda).

🔁 Alternativa: búsqueda local

Empezar con una asignación completa (inconsistente) y reparar: elegir una variable en conflicto y darle el valor que **minimiza conflictos**. Min-conflicts resuelve el problema de las n-reinas con n = 1 000 000 en tiempo casi constante de pasos esperados, pero es incompleto: puede ciclar y no puede probar insatisfacibilidad.

🧮 Ejemplo trabajado

Colorear el mapa de Australia con {R, V, A}: variables WA, NT, SA, Q, NSW, V (y T, sin restricciones); SA es adyacente a todas las continentales; además WA-NT, NT-Q, Q-NSW, NSW-V.

Backtracking con MRV + forward checking, empezando por SA (grado máximo):

```
1. SA = R          → FC: quita R de WA, NT, Q, NSW, V   (dominios: {V,A})
2. MRV empata; grado elige NT (o WA). NT = V
                   → FC: quita V de WA y Q → WA ∈ {A}, Q ∈ {A}
3. MRV: WA = A     (dominio unitario)
4. MRV: Q = A      → FC: quita A de NSW → NSW ∈ {V}
5. NSW = V         → FC: quita V de V(ictoria) → V ∈ {A}
6. V = A, T = cualquiera  ✓ solución sin un solo retroceso
```

Sin heurísticas (orden alfabético WA, NT, Q, NSW, V, SA y valores R, V, A), el mismo problema provoca retrocesos: al llegar a SA su dominio está vacío porque nadie protegió sus opciones. La diferencia no es el algoritmo sino el *orden* — esa es la lección central de los CSP.

📊 Propiedades y comparación

Método	Completo	Detecta insatisfacible	Costo típico	Cuándo usarlo
Backtracking puro	Sí	Sí	exponencial, constante alta	nunca solo; baseline
+ MRV/LCV + forward checking	Sí	Sí	exponencial, poda fuerte	CSP medianos
MAC (AC-3 en cada paso)	Sí	Sí	$O(c \cdot d^3)$ por nodo, menos nodos	restricciones densas
Min-conflicts (local)	No	No	a menudo casi lineal	n-reinas gigantes, scheduling con buena densidad de soluciones
Solver CP/SMT industrial	Sí	Sí	motores híbridos	producción

⚠️ Errores conceptuales frecuentes

1. **"AC-3 resuelve el CSP."** Solo poda dominios. Un CSP puede ser arco-consistente y no tener solución (p. ej. tres variables con dominios {R,V} y restricciones ≠ por pares). AC-3 es un filtro, la búsqueda sigue haciendo el trabajo.
2. **Confundir la dirección de MRV y LCV.** Variables: la *más* restringida primero (fallar pronto). Valores: el *menos* restrictivo primero (acomodar a los vecinos). Invertirlos degrada el rendimiento drásticamente.
3. **Tratar el CSP como búsqueda de estados atómica.** Ramificar sobre "todas las asignaciones posibles de cualquier variable" genera n!·d^n hojas donde la formulación conmutativa da d^n. La factorización es el punto.
4. **Olvidar restaurar dominios al retroceder.** Las inferencias de forward checking/MAC son por-rama; no deshacerlas corrompe silenciosamente el resto de la búsqueda.
5. **Usar min-conflicts para probar que "no hay solución".** La búsqueda local no termina con certificado de insatisfacibilidad; para eso hacen falta métodos sistemáticos.

🚀 Del aprendizaje a la operación

Un problema real de horarios o asignación de recursos rara vez se entrega como `(X, D, C)`: el trabajo duro es **modelar** — elegir variables y restricciones que el solver explote — y para producción conviene un lenguaje/solver maduro (MiniZinc, OR-Tools CP-SAT, Z3) en lugar de un backtracking propio: traen restricciones globales (`all-different` con filtrado polinómico), reinicios, aprendizaje de cláusulas y

paralelismo. Quedan además los requisitos blandos (optimización, no solo satisfacción), la explicación de infactibilidades a usuarios (núcleos IIS) y la re-resolución incremental cuando los datos cambian a mitad de ejecución.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), cap. 6 "Constraint Satisfaction Problems". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Mackworth, A. K. (1977). "Consistency in Networks of Relations". *Artificial Intelligence*, 8(1). https://doi.org/10.1016/0004-3702(77)90007-8 — uso: fuente primaria del mecanismo estudiado
- Minton, S. et al. (1992). "Minimizing conflicts: a heuristic repair method for constraint satisfaction and scheduling problems". *Artificial Intelligence*, 58(1-3). https://doi.org/10.1016/0004-3702(92)90007-K — uso: fuente primaria del mecanismo estudiado
- Rossi, F., van Beek, P. y Walsh, T. (eds.) (2006). *Handbook of Constraint Programming*. Elsevier. — uso: desarrollo extendido del tema
- MiniZinc — lenguaje de modelado de restricciones: https://www.minizinc.org/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P70 · Consistencia en redes de relaciones	1977	Convierte la propagación de restricciones en un preproceso con nombre y algoritmo: podar dominios antes de buscar, no mientras se busca.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Rossi, F., van Beek, P. y Walsh, T. — *Handbook of Constraint Programming*	2006	ISBN 9780444527264	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 6 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define problemas de satisfacción de restricciones sin usar una marca o framework como definición.
2. Explica la relación entre CSP, backtracking, consistencia, restricciones.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

019 — Lógica proposicional e inferencia

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **lógica proposicional e inferencia** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lógica proposicional e inferencia usando los conceptos `proposiciones`, `CNF`, `resolución`, `inferencia`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`proposiciones`, `CNF`, `resolución`, `inferencia`

🗺️ Ubicación en el mapa de la IA

Con la lógica, la IA pasa de *buscar* soluciones a **deducirlas**: un agente basado en conocimiento representa lo que sabe con fórmulas y deriva lo que se sigue de ellas. La lógica proposicional es el caso base — booleana, decidible, sin objetos ni relaciones — y su problema central, SAT, fue el primer problema NP-completo (Cook, 1971). Precede a la lógica de primer orden (clase 020) y alimenta directamente la industria actual: los SAT solvers descendientes de DPLL verifican hardware y software a escala, y la planificación clásica puede compilarse a SAT.

📖 Fundamentos

🔤 Sintaxis y semántica

Sintaxis: fórmulas construidas con símbolos proposicionales (P, Q, ...) y conectivas `¬, ∧, ∨, ⇒, ⇔`.
Semántica: un **modelo** (interpretación) asigna verdadero/falso a cada símbolo; las tablas de las conectivas determinan el valor de cualquier fórmula. Con `n` símbolos hay `2^n` modelos.

Conceptos pivote:

- **Satisfacible**: alguna interpretación la hace verdadera. **Válida** (tautología): todas. **Insatisfacible**: ninguna.
- **Consecuencia lógica** `KB ⊨ α`: en todo modelo donde KB es verdadera, α también. Equivalencia fundamental (refutación): `KB ⊨ α ⟺ KB ∧ ¬α es insatisfacible`.
- Un procedimiento de inferencia es **correcto** (sound) si solo deriva consecuencias, y **completo** si deriva todas.

📋 Forma normal conjuntiva (CNF)

Una fórmula está en **CNF** si es conjunción de **cláusulas** (disyunciones de literales). Toda fórmula se convierte: (1) eliminar ⇔ y ⇒ (`α ⇒ β ≡ ¬α ∨ β`), (2) empujar ¬ hacia dentro (De Morgan), (3) distribuir ∨ sobre ∧. La CNF importa porque resolución y DPLL operan sobre cláusulas.

⚔️ Resolución

Una única regla de inferencia, correcta y **refutacionalmente completa** (Robinson, 1965):

```
(ℓ1 ∨ ... ∨ ℓi ∨ ... ∨ ℓk),   (m1 ∨ ... ∨ ¬ℓi ∨ ... ∨ mn)
──────────────────────────────────────────────────────────
(ℓ1 ∨ ... ∨ ℓk ∨ m1 ∨ ... ∨ mn)      sin ℓi ni ¬ℓi, sin duplicados
```

Para probar `KB ⊨ α`: convertir `KB ∧ ¬α` a CNF y resolver pares de cláusulas hasta derivar la **cláusula vacía** □ (contradicción ⇒ se demuestra α) o hasta que no haya resolventes nuevos (⇒ no se sigue). Completa *para refutación*: no genera todas las consecuencias, pero siempre detecta la insatisfacibilidad.

⚙️ DPLL: decidir SAT con inteligencia

DPLL (Davis-Putnam-Logemann-Loveland, 1962) es backtracking sobre asignaciones parciales con dos reglas de propagación:

```
función DPLL(cláusulas, asignación):
    si alguna cláusula es falsa bajo asignación: devolver falso
    si todas las cláusulas son verdaderas: devolver verdadero
    # 1) propagación unitaria: cláusula con un solo literal libre → forzarlo
    si existe cláusula unitaria {ℓ}: devolver DPLL(cláusulas, asignación ∪ {ℓ})
    # 2) literal puro: símbolo que aparece con un solo signo → satisfacerlo
    si existe literal puro ℓ: devolver DPLL(cláusulas, asignación ∪ {ℓ})
    # 3) ramificar
    P ← elegir símbolo libre
    devolver DPLL(cláusulas, asignación ∪ {P=V}) o DPLL(cláusulas, asignación ∪ {P=F})
```

La **propagación unitaria** encadena deducciones sin ramificar (es el motor del algoritmo); es el análogo lógico del forward checking de los CSP. Los SAT solvers modernos (estilo CDCL: MiniSat, Kissat) añaden aprendizaje de cláusulas a partir de conflictos, saltos no cronológicos, reinicios y heurísticas de actividad (VSIDS), y resuelven instancias industriales de millones de variables — pese a que SAT es NP-completo, porque las instancias reales tienen estructura.

➡️ El fragmento eficiente: cláusulas de Horn

Una **cláusula de Horn** tiene a lo sumo un literal positivo (`P1 ∧ ... ∧ Pk ⇒ Q` o hechos `Q`). Sobre KB de Horn, el **encadenamiento hacia adelante** (desde los hechos, disparando reglas) y **hacia atrás** (desde la

meta, buscando soporte) son correctos, completos y de **tiempo lineal**. Este fragmento es la base de los motores de reglas de la clase 022.

🧮 Ejemplo trabajado

KB: "Si llueve, el suelo se moja" (`L ⇒ M`), "Si el suelo se moja, hay resbalones" (`M ⇒ R`), "Llueve" (`L`). Demostrar por **resolución** que `KB ⊨ R`.

1. CNF de la KB: `{¬L ∨ M}`, `{¬M ∨ R}`, `{L}`. Negación de la meta: `{¬R}`.
2. Refutación:

```
C1: ¬L ∨ M        C2: ¬M ∨ R        C3: L         C4: ¬R
C5 = res(C1, C3) sobre L :  M
C6 = res(C2, C5) sobre M :  R
C7 = res(C6, C4) sobre R :  □   (cláusula vacía → contradicción)
∴ KB ∧ ¬R es insatisfacible ⇒ KB ⊨ R  ✓
```

Con **DPLL** sobre las mismas 4 cláusulas: `{L}` es unitaria → L=V; entonces `¬L ∨ M` deja la unitaria `{M}` → M=V; entonces `{R}` unitaria → R=V; pero `{¬R}` queda falsa → insatisfacible, sin ramificar ni una vez. Toda la prueba fue propagación unitaria — típico en KB de Horn.

📊 Propiedades y comparación

Método	Correcto	Completo	Complejidad	Restricción
Tabla de verdad	Sí	Sí	O(2^n) siempre	solo viable con pocos símbolos
Resolución	Sí	Sí (refutación)	exponencial en el peor caso	requiere CNF
DPLL / CDCL	Sí	Sí (decide SAT)	exponencial peor caso; eficaz en la práctica	requiere CNF
Encadenamiento adelante/atrás	Sí	Solo en cláusulas de Horn	**lineal**	KB de Horn
Lógica proposicional en sí	—	decidible	SAT es NP-completo (Cook, 1971)	sin objetos ni cuantificadores

⚠️ Errores conceptuales frecuentes

1. **Confundir `⊨` (consecuencia semántica) con `⊢` (derivabilidad).** El primero habla de modelos; el segundo, de lo que un procedimiento deriva. Corrección y completitud son exactamente los puentes entre ambos.
2. **Leer `P ⇒ Q` como causalidad o como "si no P, no Q".** Es solo verdad funcional: falsa únicamente cuando P=V y Q=F. Con P falso, la implicación es verdadera (vacuamente).
3. **"Resolución genera todas las consecuencias de la KB."** Es completa *para refutación*: prueba cualquier consecuencia por contradicción, pero saturar la KB no enumera todo lo implicado (p. ej. nunca produce `P ∨ ¬P` desde una KB vacía).
4. **Olvidar negar la meta.** Resolver KB ∧ α y "no encontrar contradicción" no demuestra nada; la prueba es la insatisfacibilidad de KB ∧ ¬α.
5. **"SAT es NP-completo, luego es inútil en la práctica."** Los CDCL resuelven instancias industriales enormes; NP-completitud habla del peor caso, no del caso estructurado típico. La inversa también engaña: no todo se codifica eficientemente en SAT.

🚀 Del aprendizaje a la operación

Usar lógica proposicional en un sistema real (verificación de circuitos, análisis de configuraciones, planificación via SAT) exige: una **codificación** cuidadosa del dominio a variables booleanas (la transformación de Tseitin evita la explosión de la CNF a costa de variables auxiliares), un solver industrial (Kissat, Glucose, Z3) en lugar de un DPLL propio, extracción de núcleos insatisfacibles para *explicar* los fallos, y control de recursos porque el peor caso sigue siendo exponencial. El límite expresivo también pesa: sin objetos ni relaciones, cualquier dominio con individuos obliga a duplicar proposiciones (P_juan, P_maria, ...) — la motivación exacta de la clase 020.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), cap. 7 "Logical Agents". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Davis, M., Logemann, G. y Loveland, D. (1962). "A machine program for theorem-proving". *Communications of the ACM*, 5(7). https://doi.org/10.1145/368273.368557 — uso: fuente primaria del mecanismo estudiado
- Robinson, J. A. (1965). "A Machine-Oriented Logic Based on the Resolution Principle". *Journal of the ACM*, 12(1). https://doi.org/10.1145/321250.321253 — uso: fuente primaria del mecanismo estudiado

- Cook, S. A. (1971). "The complexity of theorem-proving procedures". *Proc. STOC '71*. https://doi.org/10.1145/800157.805047 — uso: fuente primaria del mecanismo estudiado
- Biere, A., Heule, M., van Maaren, H. y Walsh, T. (eds.) (2021). *Handbook of Satisfiability* (2.ª ed.). IOS Press. — uso: desarrollo extendido del tema

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P65 · Un programa de máquina para demostración de teoremas	1962	El algoritmo que sigue siendo el esqueleto de todo solucionador SAT moderno: propagar primero, ramificar solo cuando no queda deducción por hacer.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Biere, A. et al. — *Handbook of Satisfiability*	2021	ISBN 9781643681603	citada en las referencias de esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 7 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define lógica proposicional e inferencia sin usar una marca o framework como definición.

2. Explica la relación entre proposiciones, CNF, resolución, inferencia.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

020 — Lógica de primer orden y unificación

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **lógica de primer orden y unificación** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lógica de primer orden y unificación usando los conceptos `predicados`, `cuantificadores`, `unificación`, `sustitución`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`predicados`, `cuantificadores`, `unificación`, `sustitución`

🗺️ Ubicación en el mapa de la IA

La lógica de primer orden (FOL) añade a la proposicional lo que a esta le falta: **objetos, relaciones y cuantificadores**. Es el lenguaje de representación más influyente de la IA simbólica: Prolog es un fragmento ejecutable de FOL, las ontologías de la clase 021 son fragmentos decidibles de FOL, y STRIPS (clase 023) describe acciones con literales de primer orden. El mecanismo técnico que la hace computable — la **unificación** de Robinson (1965) — reapareció en la inferencia de tipos de los lenguajes funcionales (Hindley-Milner) y en el pattern matching moderno.

📖 Fundamentos

🌎 Sintaxis y semántica

Elementos del lenguaje:

- **Términos** (denotan objetos): constantes (`Juan`), variables (`x`), funciones (`PadreDe(Juan)`).
- **Fórmulas atómicas** (denotan hechos): predicados sobre términos, `Hermano(Juan, Ricardo)`.
- **Conectivas** proposicionales y **cuantificadores**: `∀x` (universal), `∃x` (existencial).

Un **modelo** de FOL tiene un dominio de objetos y una interpretación que asigna a cada constante un objeto, a cada función una función sobre el dominio y a cada predicado una relación. `KB ⊨ α` se define igual que en proposicional, pero ahora los modelos pueden ser infinitos.

Patrones de traducción que hay que dominar (y donde más se falla):

```
"Todos los reyes son personas":   ∀x  Rey(x) ⇒ Persona(x)      (∀ con ⇒)
"Algún rey es cruel":             ∃x  Rey(x) ∧ Cruel(x)        (∃ con ∧)
Dualidad:                          ¬∃x P(x)  ≡  ∀x ¬P(x)
```

Usar `∧` con `∀` afirma que *todo objeto* es rey y persona; usar `⇒` con `∃` se satisface con cualquier no-rey: ambas combinaciones son casi siempre un error de modelado.

🔁 De la instanciación a la unificación

La inferencia ingenua **proposicionaliza**: instanciación universal (sustituir `∀x` por cada término concreto) reduce FOL a proposicional. Funciona (Herbrand, 1930) pero explota: con funciones, el conjunto de términos es infinito; la semidecidibilidad de FOL (Church-Turing, 1936) significa que si `KB ⊨ α` existe prueba finita, pero si no, el procedimiento puede no terminar jamás.

La **unificación** evita instanciar a ciegas: encuentra la sustitución que hace idénticas dos expresiones.

```
UNIFICAR(Conoce(Juan, x), Conoce(Juan, Ana))    = {x/Ana}
UNIFICAR(Conoce(Juan, x), Conoce(y, Madre(y)))  = {y/Juan, x/Madre(Juan)}
UNIFICAR(Conoce(Juan, x), Conoce(x, Elena))     = fallo (x no puede ser Juan y Elena)
                                                    → renombrar variables: con Conoce(z, Elena) sí: {z/Juan,
x/Elena}
UNIFICAR(P(x), P(F(x)))                          = fallo por OCCURS-CHECK (x dentro de F(x))
```

El algoritmo recorre ambas expresiones en paralelo componiendo sustituciones y devuelve el **unificador más general** (MGU), único salvo renombramiento: el que compromete lo mínimo. El **occurs-check** (¿aparece la variable dentro del término con que se unifica?) evita términos infinitos; muchos Prolog lo omiten por eficiencia, sacrificando corrección en casos límite.

⚙️ Inferencia con reglas: Modus Ponens Generalizado

Para KB en **cláusulas definidas** (`p1 ∧ ... ∧ pn ⇒ q` con literales positivos):

```
p1', ..., pn',   (p1 ∧ ... ∧ pn ⇒ q)        con θ tal que pi'θ = piθ para todo i
──────────────────────────────────
             qθ
```

- **Encadenamiento hacia adelante**: desde los hechos, aplicar reglas cuyas premisas unifican, hasta derivar la meta o saturar. Correcto y completo para cláusulas definidas (sin funciones: termina; es la semántica de Datalog).
- **Encadenamiento hacia atrás**: desde la meta, buscar reglas cuya conclusión unifique con ella y perseguir sus premisas como submetas (DFS). Es el motor de Prolog; puede entrar en bucles infinitos con recursión izquierda.

⚔️ Resolución de primer orden

Generaliza la resolución proposicional: convierte a CNF (con **skolemización**: `∃` se reemplaza por constantes/funciones de Skolem dependientes de los `∀` que lo dominan) y resuelve cláusulas cuyos literales complementarios **unifican**, aplicando el MGU al resolvente. Es refutacionalmente completa para FOL (Robinson, 1965); es la base de los demostradores automáticos (Vampire, E) que hoy ganan la competición CASC.

🧮 Ejemplo trabajado

KB (el clásico "el criminal" de AIMA, abreviado):

```
R1: Americano(x) ∧ Arma(y) ∧ Vende(x, y, z) ∧ Hostil(z) ⇒ Criminal(x)
H1: Americano(West)          H2: Misil(M1)         H3: Posee(Nono, M1)
R2: Misil(y) ∧ Posee(Nono, y) ⇒ Vende(West, y, Nono)
R3: Misil(y) ⇒ Arma(y)       R4: Enemigo(z, America) ⇒ Hostil(z)
H4: Enemigo(Nono, America)
```

Encadenamiento hacia adelante, ronda a ronda:

```
Ronda 1:
  R3 con {y/M1}  (H2)            → Arma(M1)
  R2 con {y/M1}  (H2, H3)        → Vende(West, M1, Nono)
  R4 con {z/Nono} (H4)           → Hostil(Nono)
Ronda 2:
  R1 con {x/West, y/M1, z/Nono}  (H1, Arma(M1), Vende(...), Hostil(Nono))
                                 → Criminal(West)  ✓
```

Hacia atrás desde `Criminal(West)`: unifica con la conclusión de R1 vía `{x/West}`; submetas `Americano(West)` ✓ (H1), `Arma(y)` → R3 → `Misil(y)` → `{y/M1}` ✓, `Vende(West, M1, z)` → R2 → `{z/Nono}` ✓, `Hostil(Nono)` → R4 ✓. Cada paso es una unificación verificable a mano; nótese cómo las sustituciones se **componen** y propagan entre submetas (la `y` de Arma queda ligada a M1 para Vende).

📊 Propiedades y comparación

Propiedad	Lógica proposicional	FOL (cláusulas definidas)	FOL completa
Expresa objetos/relaciones	No	Sí	Sí
Cuantificadores	No	∀ implícito en reglas	∀ y ∃
Decidible	Sí (NP-completo)	Datalog: sí; con funciones: no	No: semidecidible
Motor típico	DPLL/CDCL	encadenamiento + unificación (Prolog)	resolución + skolemización (Vampire)
Costo de la expresividad	duplicar símbolos por individuo	bucles posibles hacia atrás	no-terminación posible

⚠️ Errores conceptuales frecuentes

1. **`∀` con `∧` y `∃` con `⇒`.** "∀x Rey(x) ∧ Persona(x)" dice que todo el universo es rey; "∃x Rey(x) ⇒ Cruel(x)" es verdadera en cuanto exista un no-rey. Las combinaciones correctas son ∀-⇒ y ∃-∧.
2. **Unificar sin renombrar variables.** `Conoce(Juan, x)` y `Conoce(x, Elena)` comparten `x` por accidente sintáctico; cada cláusula debe llevar variables frescas (standardizing apart) antes de unificar.
3. **Omitir el occurs-check y no saberlo.** `P(x)` con `P(F(x))` no unifica; Prolog estándar lo acepta y crea un término cíclico. Es un trade-off de eficiencia que hay que conocer, no ignorar.
4. **Creer que skolemizar preserva equivalencia.** Preserva *satisfacibilidad* (que es lo que la refutación necesita), no equivalencia lógica: `∃x P(x)` y `P(C_sk)` no son equivalentes.
5. **Esperar que la inferencia FOL siempre termine.** FOL es semidecidible: la búsqueda de prueba de algo que no se sigue puede correr para siempre. Todo sistema real necesita límites de recursos y estrategias de terminación.

🚀 Del aprendizaje a la operación

Para usar FOL en producción hay que elegir el fragmento con juicio: Datalog para consultas recursivas sobre bases de datos (garantiza terminación), Prolog para prototipos de razonamiento con control del programador (cortes, negación por fallo — que es *supuesto de mundo cerrado*, no negación clásica), demostradores tipo Vampire/E para verificación, o lógicas de descripción (clase 021) cuando se necesita decidibilidad. Quedan además la ingeniería del conocimiento (¿quién escribe y mantiene los axiomas?), la indexación de términos para unificar contra millones de cláusulas y la integración con datos ruidosos, donde la lógica pura no ofrece gradación de incertidumbre.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;

- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), caps. 8-9 "First-Order Logic" e "Inference in First-Order Logic". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Robinson, J. A. (1965). "A Machine-Oriented Logic Based on the Resolution Principle". *Journal of the ACM*, 12(1). https://doi.org/10.1145/321250.321253 — uso: fuente primaria del mecanismo estudiado
- Kowalski, R. (1974). "Predicate Logic as Programming Language". *IFIP Congress* — el puente entre FOL y Prolog.
- SWI-Prolog — implementación libre de referencia: https://www.swi-prolog.org/ — uso: referencia consultada en su fuente original
- Stanford Encyclopedia of Philosophy — "Classical Logic": https://plato.stanford.edu/entries/logic-classical/ — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P66 · Una lógica orientada a máquina basada en el principio de resolución	1965	Reduce toda la inferencia de primer orden a una sola regla, y hace la unificación computable con el unificador más general.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · caps. 8-9 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define lógica de primer orden y unificación sin usar una marca o framework como definición.
2. Explica la relación entre predicados, cuantificadores, unificación, sustitución.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

021 — Representación del conocimiento y ontologías

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **representación del conocimiento y ontologías** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar representación del conocimiento y ontologías usando los conceptos `ontologías`, `taxonomías`, `relaciones`, `inferencia`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`ontologías`, `taxonomías`, `relaciones`, `inferencia`

🗺️ Ubicación en el mapa de la IA

Si las clases 019-020 dieron el *lenguaje* (lógica), esta clase trata el *contenido*: qué conceptos, categorías y relaciones hay que escribir para que un sistema razone sobre un dominio. La ingeniería ontológica viene de las redes semánticas (Quillian, 1968) y los marcos (Minsky, 1974), maduró en las lógicas de descripción y cristalizó en estándares web (RDF, OWL). Hoy sostiene los grafos de conocimiento de Google y Wikidata, y es la mitad simbólica de los sistemas neuro-simbólicos del proyecto de la clase 024.

📖 Fundamentos

🏛️ Ontología: definición operativa

Una **ontología** es una especificación explícita y formal de una conceptualización compartida (Gruber, 1993): define **clases** (categorías), **individuos** (instancias), **relaciones** (propiedades entre objetos), **atributos** y **axiomas** que restringen las interpretaciones válidas. Una **taxonomía** es el caso particular donde solo hay jerarquía de subsunción ($\sqsubseteq$, "es-un").

Distinción estructural clave:

- **TBox** (terminológico): conocimiento sobre conceptos — `Perro ⊑ Mamífero`, `Mamífero ⊓ Ave ⊑ ⊥` (disjuntos).
- **ABox** (asercional): conocimiento sobre individuos — `Perro(Fido)`, `dueño(Fido, Ana)`.

Redes semánticas y herencia

Una red semántica es un grafo: nodos = conceptos/individuos, aristas = relaciones tipadas (`es-un`, `parte-de`, `tiene`). Su operación central es la **herencia**: las propiedades fluyen hacia abajo por `es-un`. La herencia **por defecto con excepciones** (los pájaros vuelan; los pingüinos son pájaros que no vuelan) exige razonamiento **no monótono**: añadir información (Piolín es pingüino) puede *retirar* conclusiones (ya no vuela). La lógica clásica es monótona, por eso se desarrollaron formalismos específicos (lógica por defecto de Reiter, circunscripción de McCarthy).

```
HEREDAR(individuo, propiedad):
    valor ← buscar propiedad localmente en individuo
    si existe: devolver valor                    # la excepción gana
    para cada clase C en cadena es-un (de específica a general):
        si C define propiedad: devolver C.valor   # el más específico gana
    devolver desconocido
```

🔬 Lógicas de descripción: expresividad con decidibilidad

Las **description logics** (DL) son fragmentos de FOL diseñados para que la inferencia sea **decidible** con complejidad conocida. Constructores típicos: `⊓` (intersección), `⊔` (unión), `¬`, `∃r.C` ("algún r lleva a un C"), `∀r.C`, cardinalidades. Servicios de razonamiento estándar:

- **Subsunción**: ¿`C ⊑ D` se sigue de la TBox? (clasificar la jerarquía completa).
- **Satisfacibilidad de concepto**: ¿puede `C` tener instancias?
- **Realización**: ¿de qué clases es instancia cada individuo de la ABox?
- **Consistencia** global de TBox + ABox.

OWL 2 DL (estándar W3C) corresponde a la DL *SROIQ*; sus perfiles (EL, QL, RL) recortan expresividad a cambio de razonamiento polinómico — OWL 2 EL es el perfil de SNOMED CT, la ontología clínica de ~350 000 conceptos que se clasifica en minutos con razonadores tipo ELK.

🧱 Decisiones de diseño recurrentes

- **Clase vs. individuo**: ¿"Águila" es una clase (cada águila concreta) o un individuo (la especie, en una ontología de taxonomía biológica)? Depende del uso; mezclarlos es fuente clásica de incoherencias (OWL 2 permite *punning* controlado).
- **`es-un` vs. `parte-de`**: la subsunción hereda propiedades; la mereología no (el motor es parte del coche; el coche no hereda "gira a 3000 rpm").
- **Mundo abierto vs. cerrado**: OWL asume **mundo abierto** (lo no afirmado es *desconocido*, no falso), al contrario que las bases de datos y Prolog. Cambia radicalmente qué se puede concluir: que la ontología no diga que Ana tiene hijos no permite inferir que no los tiene.
- **Reutilizar vs. construir**: ontologías superiores (BFO, DOLCE) y de dominio (SNOMED, Gene Ontology, schema.org) existen para no partir de cero.

🧮 Ejemplo trabajado

Mini-ontología zoológica (TBox + ABox) y las inferencias que un razonador DL extrae:

```
TBox:
  A1: Pinguino ⊑ Ave                A2: Ave ⊑ Vertebrado
  A3: Ave ⊑ Ovíparo                 A4: Mamifero ⊑ Vertebrado
  A5: Ave ⊓ Mamifero ⊑ ⊥             (disjuntos)
  A6: Volador ≡ Animal ⊓ ∃medio.Aire
ABox:
  F1: Pinguino(Piolin)              F2: Mamifero(Rex)
```

Inferencias paso a paso:

```
1. Subsunción transitiva:  Pinguino ⊑ Ave ⊑ Vertebrado  ⇒  Pinguino ⊑ Vertebrado
2. Realización:            F1 + A1 ⇒ Ave(Piolin);  + A2 ⇒ Vertebrado(Piolin)
                           F1 + A3 ⇒ Ovíparo(Piolin)
3. Consistencia:           si alguien añade Mamifero(Piolin), A5 hace la ABox
                           inconsistente ⇒ el razonador lo rechaza con explicación
4. Mundo abierto:          ¿Volador(Piolin)? DESCONOCIDO — nada afirma ∃medio.Aire,
                           pero tampoco lo niega. Para negar hace falta el axioma
                           Pinguino ⊑ ¬Volador (así se modela la excepción en DL,
                           que no tiene defaults: la clase excepcional se excluye
                           explícitamente).
```

El punto 4 es el contraste práctico entre herencia por defecto (redes semánticas) y DL: en DL las "excepciones" se modelan con axiomas de exclusión explícitos, conservando la monotonía.

📊 Propiedades y comparación

Formalismo	Expresividad	Inferencia	Decidible	Uso típico
Taxonomía / tesauro (SKOS)	jerarquía + sinónimos	transitividad	Sí (trivial)	catálogos, vocabularios
Red semántica / marcos	relaciones + defaults	herencia con excepciones	según formalización	prototipos, UX de conocimiento
RDF + RDFS	tripletas + subclases	reglas simples	Sí (P)	grafos de conocimiento, linked data
OWL 2 DL (SROIQ)	alta	tableaux/consecuencia	Sí (N2EXPTIME)	ontologías ricas (biomedicina)
OWL 2 EL	media	saturación	Sí (PTIME)	SNOMED CT, ontologías enormes
FOL completa	máxima	demostración de teoremas	No	verificación, matemáticas

⚠️ Errores conceptuales frecuentes

1. **Confundir `es-un` (subsunción) con `instancia-de`.** "Fido es-un Perro" es pertenencia (ABox); "Perro es-un Mamífero" es inclusión de clases (TBox). Colapsarlos produce herencias sin sentido ("Fido es una subclase").
2. **Modelar `parte-de` como subsunción.** La rueda no es un tipo de coche. La mereología necesita relaciones propias (transitivas, pero sin herencia de propiedades arbitrarias).
3. **Leer OWL con mentalidad de base de datos (mundo cerrado).** La ausencia de un hecho no lo hace falso; las restricciones de cardinalidad no "validan datos faltantes" como un esquema SQL, y esa confusión es la queja n.º 1 de los recién llegados a OWL.
4. **Esperar defaults y excepciones de la lógica clásica.** DL/OWL son monótonas: "los pájaros vuelan, salvo los pingüinos" exige remodelar (clase AveVoladora, o axioma de exclusión), no un default que se retracta.
5. **Construir la ontología por los nombres y no por los axiomas.** Llamar a una clase "ClienteImportante" no le da semántica; sin axiomas que la definan, el razonador no puede clasificar nada en ella.

🚀 Del aprendizaje a la operación

Una ontología operativa es un artefacto de software con ciclo de vida: control de versiones y revisión por expertos de dominio, pruebas de regresión de inferencias (¿esta edición reclasificó 400 conceptos sin querer?), razonadores dimensionados al perfil elegido (ELK para EL, HermiT para DL completa), y pipelines de poblado de la ABox desde datos reales — hoy, a menudo con extracción por LLM *validada* contra los axiomas. El costo dominante no es el razonador sino el **mantenimiento del consenso**: una ontología es un acuerdo social formalizado, y sin gobernanza se degrada en meses.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell, S. y Norvig, P. (2021). *AIMA* (4.ª ed.), cap. 10 "Knowledge Representation". https://aima.cs.berkeley.edu/ — uso: desarrollo extendido del tema
- Gruber, T. R. (1993). "A translation approach to portable ontology specifications". *Knowledge Acquisition*, 5(2). https://doi.org/10.1006/knac.1993.1008 — uso: fuente primaria del mecanismo estudiado
- Baader, F. et al. (eds.) (2007). *The Description Logic Handbook* (2.ª ed.). Cambridge University Press. — uso: desarrollo extendido del tema
- W3C (2012). *OWL 2 Web Ontology Language — Overview* (2.ª ed.): https://www.w3.org/TR/owl2-overview/ — uso: marco normativo de referencia
- Minsky, M. (1974). "A Framework for Representing Knowledge". MIT AI Memo 306. https://dspace.mit.edu/handle/1721.1/6089 — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P71 · Un enfoque de traducción para especificaciones de ontologías portables	1993	Da la definición que se sigue citando —una ontología es una especificación explícita de una conceptualización— y cinco criterios para juzgarla.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 10 · obra de referencia de la parte 01
Baader y F. et al — *The Description Logic Handbook*	2007	ISBN 9781280417993	citada en las referencias de esta clase
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

Normas y documentación oficial que aplica esta clase: https://www.w3.org/TR/owl2-overview/

📝 Evaluación completa

❓ Preguntas

1. Define representación del conocimiento y ontologías sin usar una marca o framework como definición.
2. Explica la relación entre ontologías, taxonomías, relaciones, inferencia.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

022 — Sistemas expertos y motores de reglas

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `logic` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **sistemas expertos y motores de reglas** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sistemas expertos y motores de reglas usando los conceptos `reglas`, `encadenamiento`, `explicación`, `conocimiento`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`reglas`, `encadenamiento`, `explicación`, `conocimiento`

🗺️ Ubicación en el mapa de la IA

Los sistemas expertos fueron el primer éxito comercial de la IA: en los años 70-80, sistemas como DENDRAL, MYCIN y XCON demostraron que el conocimiento de un especialista podía codificarse en reglas y ejecutarse con un motor de inferencia. Se apoyan directamente en la lógica de las clases 019-021 (los hechos y reglas son fórmulas, la inferencia es modus ponens aplicado sistemáticamente). Habilitan después la planificación clásica (clase 023, donde las "reglas" pasan a ser operadores con precondiciones y efectos) y son el componente simbólico canónico de los sistemas neuro-simbólicos del proyecto de la clase 024. Los motores de reglas modernos (Drools, CLIPS, motores de decisión bancarios) descienden en línea directa de esta arquitectura.

📖 Fundamentos

🏛️ Arquitectura de un sistema experto

Un **sistema experto** separa el conocimiento del mecanismo que lo usa. Sus componentes clásicos:

1. **Base de conocimiento**: reglas de producción `SI condiciones ENTONCES conclusión` escritas por (o con) expertos del dominio.
2. **Memoria de trabajo**: los hechos conocidos en un instante dado; crece durante la inferencia.
3. **Motor de inferencia**: el algoritmo que decide qué reglas aplicar y en qué orden. Es genérico: el mismo motor sirve para medicina o configuración de hardware.
4. **Componente de explicación**: responde *¿por qué?* (qué regla pide este dato) y *¿cómo?* (qué cadena de reglas produjo esta conclusión).
5. **Interfaz de adquisición de conocimiento**: cómo se añaden y mantienen las reglas.

Esta separación es la tesis central de la época: *"In the knowledge lies the power"* (Feigenbaum). El motor es trivial comparado con el costo de capturar y mantener el conocimiento.

➡️ Encadenamiento hacia adelante (dirigido por datos)

El **forward chaining** parte de los hechos y dispara reglas hasta el punto fijo. Ciclo *match-resolve-act*:

```
mientras haya cambios:
    MATCH:   encontrar todas las reglas cuyas condiciones ⊆ memoria de trabajo
             y cuya conclusión aún no esté afirmada   → conjunto de conflicto
    RESOLVE: elegir una regla del conjunto de conflicto
             (estrategias: especificidad, recencia, prioridad explícita)
    ACT:     añadir la conclusión a la memoria de trabajo
             y registrar la regla disparada (para la explicación)
```

Es **completo para cláusulas de Horn**: deriva todo hecho atómico implicado por la base (es exactamente el algoritmo de la clase 019, ahora con arquitectura alrededor). Conviene cuando llegan datos y se quiere ver *todo* lo que se sigue de ellos (monitorización, configuración).

⬅️ Encadenamiento hacia atrás (dirigido por objetivos)

El **backward chaining** parte de una hipótesis y busca reglas que la concluyan, convirtiendo sus condiciones en subobjetivos, recursivamente, hasta llegar a hechos conocidos o preguntas al usuario. Es el mecanismo de Prolog y de MYCIN. Conviene cuando hay *una* pregunta concreta y muchos hechos irrelevantes: evita derivar conclusiones que nadie pidió. Riesgo: ciclos entre reglas (`a → b`, `b → a`) exigen detección de objetivos repetidos en la pila.

⚡ El algoritmo Rete

El paso MATCH ingenuo re-evalúa todas las reglas contra toda la memoria en cada ciclo: O(reglas × hechos^condiciones). **Rete** (Forgy, 1982) lo evita compilando las condiciones de todas las reglas en una **red de discriminación**:

- **Nodos alfa**: filtran hechos por condiciones de un solo patrón (p. ej. `tipo = cliente`); su resultado se guarda en *memorias alfa*.
- **Nodos beta**: hacen *joins* incrementales entre memorias (comparten variables entre condiciones); su resultado se guarda en *memorias beta*.
- Cuando un hecho entra o sale de la memoria de trabajo, solo se propaga **el cambio** (delta) por la red; las coincidencias previas quedan memorizadas.

Trade-off explícito: Rete cambia memoria por velocidad — las memorias alfa/beta pueden crecer mucho, pero cada ciclo cuesta proporcionalmente al cambio, no al total. Es la base de OPS5, CLIPS y Drools (variante ReteOO/PHREAK).

🌡️ Incertidumbre: factores de certeza de MYCIN

MYCIN (Shortliffe, años 70) diagnosticaba infecciones bacterianas con ~600 reglas y necesitaba grados de creencia. Introdujo el **factor de certeza** `CF ∈ [-1, 1]` (−1 = refutado, 0 = sin evidencia, +1 = confirmado), con reglas de combinación:

```
Encadenamiento (regla con CF_regla y premisa con CF_premisa > 0):
    CF(conclusión) = CF_regla × CF_premisa

Premisas conjuntas:  CF(p1 ∧ p2) = min(CF(p1), CF(p2))

Evidencia paralela (dos reglas concluyen lo mismo, ambos CF > 0):
    CF_comb = CF1 + CF2 × (1 − CF1)
```

Los CF **no son probabilidades**: la combinación asume independencia de las evidencias y solo aproxima un razonamiento bayesiano bajo condiciones restrictivas (Heckerman, 1986). Aun así, MYCIN alcanzó desempeño comparable al de especialistas en evaluaciones ciegas — y nunca se usó clínicamente, por responsabilidad legal e integración, una lección de despliegue tan importante como el algoritmo.

🧮 Ejemplo trabajado

1) Forward chaining con la base del laboratorio. Hechos iniciales: `{tiene_datos, tiene_objetivo}`. Reglas: R1 `{tiene_datos, tiene_objetivo} → puede_experimentar`, R2 `{puede_experimentar} → requiere_baseline`, R3 `{requiere_baseline} → requiere_evaluacion`.

```
Ciclo 1: MATCH → {R1}           ACT → añade puede_experimentar
Ciclo 2: MATCH → {R2}           ACT → añade requiere_baseline
Ciclo 3: MATCH → {R3}           ACT → añade requiere_evaluacion
Ciclo 4: MATCH → ∅              punto fijo alcanzado
```

Memoria final = `{tiene_datos, tiene_objetivo, puede_experimentar, requiere_baseline, requiere_evaluacion}`: 2 hechos iniciales + 3 derivados = 5 hechos, con 3 reglas disparadas. La lista `rules_fired` del laboratorio ES el componente de explicación: cada conclusión conserva la regla que la originó.

2) Factores de certeza. Dos reglas concluyen `infección_por_pseudomonas`: R_a con CF 0,6 cuya premisa se cree con CF 0,8; R_b con CF 0,5 cuya premisa se cree con CF 1,0.

```
CF_a = 0,6 × 0,8 = 0,48
CF_b = 0,5 × 1,0 = 0,50
CF_comb = 0,48 + 0,50 × (1 − 0,48) = 0,48 + 0,26 = 0,74
```

Dos evidencias moderadas producen una creencia alta, sin llegar nunca a 1: ese es el comportamiento diseñado de la combinación paralela.

📊 Propiedades y comparación

Criterio	Forward chaining	Backward chaining	Forward + Rete
Dirección	datos → conclusiones	objetivo → datos	datos → conclusiones
Deriva	todo el punto fijo	solo lo necesario para el objetivo	todo el punto fijo
Costo por ciclo	O(reglas × hechos) ingenuo	proporcional a la prueba	proporcional al **cambio**
Memoria	baja	pila de objetivos	alta (memorias alfa/beta)
Uso típico	monitorización, configuración	diagnóstico, consulta	motores de reglas de producción
Completitud (Horn)	sí	sí (con manejo de ciclos)	sí

⚠️ Errores conceptuales frecuentes

1. **"El motor de inferencia contiene el conocimiento."** Falso: el motor es genérico; el conocimiento vive en las reglas. Por eso el cuello de botella histórico fue la *adquisición de conocimiento*, no la inferencia.
2. **Confundir factores de certeza con probabilidades.** Los CF no obedecen los axiomas de probabilidad; su combinación asume independencia. Con evidencias correlacionadas, los CF sobreestiman la certeza — la respuesta rigurosa son las redes bayesianas (parte 02).
3. **"Forward y backward chaining derivan cosas distintas."** Sobre cláusulas de Horn ambos son correctos y completos para hechos atómicos; difieren en *qué* trabajo hacen (todo el punto fijo vs. lo necesario para un objetivo), no en la validez.
4. **Creer que Rete es un algoritmo de inferencia distinto.** Rete solo optimiza el paso MATCH del forward chaining; las conclusiones son idénticas a las del algoritmo ingenuo.
5. **"Más reglas = mejor sistema."** Las bases grandes sufren interacciones no previstas entre reglas (una regla nueva dispara cadenas inesperadas); sin suites de pruebas de regresión sobre casos, el mantenimiento colapsa — la causa principal de la caída comercial de los sistemas expertos.

🚀 Del aprendizaje a la operación

El laboratorio dispara 3 reglas escritas a mano sobre 2 hechos; un motor de decisión real (crédito, tarificación, elegibilidad) maneja miles de reglas con ciclo de vida propio: versionado y gobernanza de reglas (quién aprueba un cambio), pruebas de regresión sobre carteras históricas de casos, resolución de conflictos auditable y trazas de explicación que satisfagan a un regulador, no solo a un desarrollador. Falta además el manejo de incertidumbre honesto (los CF del ejemplo no sobreviven evidencia correlacionada) y la integración

con datos vivos: en producción, el paso caro no es inferir sino mantener la base de reglas alineada con una realidad que cambia.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("logic")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell & Norvig, *Artificial Intelligence: A Modern Approach* 4e — cap. 9 (inferencia con reglas, forward/backward chaining) — uso: desarrollo extendido del tema
- Forgy, C. (1982). "Rete: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem". *Artificial Intelligence* 19(1) — uso: fuente primaria del mecanismo estudiado
- Buchanan & Shortliffe (1984). *Rule-Based Expert Systems: The MYCIN Experiments* — libro completo gratuito — uso: referencia consultada en su fuente original
- CLIPS — motor de reglas de dominio público (documentación oficial) — uso: referencia consultada en su fuente original
- Drools — documentación oficial del motor de reglas (Rete/PHREAK) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P69 · Un modelo de razonamiento inexacto en medicina	1975	El motor de MYCIN: razonar con grados de creencia y explicar cada conclusión por las reglas que la sostienen.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 9 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define sistemas expertos y motores de reglas sin usar una marca o framework como definición.
2. Explica la relación entre reglas, encadenamiento, explicación, conocimiento.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.

- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

023 — Planificación clásica con STRIPS y PDDL

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 4
Laboratorio: `workflow` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **planificación clásica con strips y pddl** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar planificación clásica con strips y pddl usando los conceptos `STRIPS`, `PDDL`, `precondiciones`, `efectos`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`STRIPS`, `PDDL`, `precondiciones`, `efectos`

🗺️ Ubicación en el mapa de la IA

La planificación clásica une las dos mitades de esta parte del curso: la búsqueda (clases 013-017) aporta el algoritmo y la lógica (clases 019-021) aporta la representación. En lugar de enumerar estados atómicos, STRIPS (Fikes y Nilsson, 1971, para el robot Shakey) los describe como conjuntos de literales y describe las acciones por sus precondiciones y efectos — con lo que un solo dominio compacto genera espacios de estados astronómicos. PDDL (1998) estandarizó esa notación y creó la Competición Internacional de Planificación (IPC), que impulsó los planificadores modernos (Fast Downward, LAMA). La planificación es hoy el componente deliberativo de robots, logística y orquestación, y el contraste directo con los agentes LLM que "planifican" sin garantías formales.

📖 Fundamentos

📐 El modelo de planificación clásica

La planificación clásica asume un entorno **determinista, completamente observable, estático y discreto**. Un problema se define como `⟨P, A, s0, G⟩`:

- `P`: conjunto de **predicados** (hechos atómicos posibles).
- `s0 ⊆ P`: estado inicial — el conjunto de hechos verdaderos al comienzo. Todo lo no listado es falso (**supuesto de mundo cerrado**).
- `G`: la meta, un conjunto de literales que deben ser verdaderos al final (cualquier estado que los contenga sirve).
- `A`: acciones en formato **STRIPS**, cada una con tres listas:

```
Acción:       mover(b, x, y)        # mover bloque b desde x hasta y
PRE  (precondiciones): {sobre(b,x), libre(b), libre(y)}
ADD  (efectos positivos): {sobre(b,y), libre(x)}
DEL  (efectos negativos): {sobre(b,x), libre(y)}
```

La semántica de la transición es puramente conjuntista:

```
aplicable(a, s)  ⇔  PRE(a) ⊆ s
RESULT(s, a)     =   (s \ DEL(a)) ∪ ADD(a)
```

Este es el mismo `RESULT` de la clase 013, pero ahora **descrito con lógica en vez de programado**: la "suposición STRIPS" dice que todo lo no mencionado en ADD/DEL permanece igual, lo que resuelve pragmáticamente el *problema del marco* (frame problem) de la clase 021.

📄 PDDL: el lenguaje estándar

PDDL separa lo reutilizable de lo concreto:

- **Domain**: predicados y esquemas de acción con variables (`mover(?b ?x ?y)`).
- **Problem**: objetos concretos, estado inicial `(:init ...)` y meta `(:goal ...)`.

```
(:action mover
  :parameters (?b ?x ?y)
  :precondition (and (sobre ?b ?x) (libre ?b) (libre ?y))
  :effect (and (sobre ?b ?y) (libre ?x)
               (not (sobre ?b ?x)) (not (libre ?y))))
```

Extensiones por niveles (declaradas con `:requirements`): tipos, precondiciones negativas, costos de acción (`:action-costs`), efectos condicionales, PDDL2.1 añade tiempo y variables numéricas. Un planificador anuncia qué requisitos soporta; la referencia viva es planning.wiki.

🔎 Cómo se resuelve: búsqueda + heurísticas independientes del dominio

Los planificadores actuales dominantes hacen **búsqueda hacia adelante en el espacio de estados** (A*, greedy best-first) con heurísticas extraídas* automáticamente del dominio — la diferencia clave con la clase 016, donde la heurística la diseñaba un humano:

- **Relajación por borrado (delete relaxation)**: ignorar las listas DEL. En el problema relajado los hechos solo se acumulan, y estimar el costo es tratable. De ahí salen `h_max` (admisible, débil), `h_add` (informativa, no admisible) y `h_FF` (longitud de un plan relajado).
- **Abstracciones y landmarks**: proyectar el problema a subconjuntos de variables (pattern databases) o detectar hechos que *todo* plan debe lograr (landmarks, base del planificador LAMA).

GraphPlan (Blum y Furst, 1997) fue el hito intermedio: construye un **grafo de planificación** de niveles alternos de hechos y acciones, propagando relaciones de **exclusión mutua (mutex)** — dos acciones son mutex si una borra precondiciones o efectos de la otra; dos hechos son mutex si toda forma de producirlos es mutex. La meta es alcanzable como muy pronto en el primer nivel donde todos sus literales aparecen sin mutex entre sí; luego se extrae el plan hacia atrás. Hoy GraphPlan casi no se usa como planificador, pero su grafo sigue vivo como fuente de heurísticas.

🧨 Complejidad

Decidir si existe un plan (PLANSAT) es **PSPACE-completo** (Bylander, 1994) incluso para STRIPS proposicional. Las heurísticas no eliminan esa cota: la desplazan — por eso el campo se evalúa empíricamente en la IPC con dominios de referencia, no con garantías universales.

🧮 Ejemplo trabajado

Mundo de bloques. Tres bloques; estado inicial: C sobre A, A y B sobre la mesa. Meta: `{sobre(A,B), sobre(B,C)}`.

```
s0 = {sobre(C,A), sobreMesa(A), sobreMesa(B), libre(C), libre(B)}

Paso 1: moverAMesa(C, A)
  PRE {sobre(C,A), libre(C)} ⊆ s0 ✓
  s1 = {sobreMesa(A), sobreMesa(B), sobreMesa(C), libre(A), libre(B), libre(C)}

Paso 2: moverDesdeMesa(B, C)
  PRE {sobreMesa(B), libre(B), libre(C)} ⊆ s1 ✓
  s2 = {sobreMesa(A), sobreMesa(C), sobre(B,C), libre(A), libre(B)}

Paso 3: moverDesdeMesa(A, B)
  PRE {sobreMesa(A), libre(A), libre(B)} ⊆ s2 ✓
  s3 = {sobreMesa(C), sobre(B,C), sobre(A,B), libre(A)}

G = {sobre(A,B), sobre(B,C)} ⊆ s3 ✓  →  plan de 3 acciones, óptimo.
```

Obsérvese la trampa clásica: si se intenta lograr `sobre(A,B)` *primero* (mover A sobre B de inmediato), la meta parcial debe deshacerse para lograr `sobre(B,C)` — las metas interactúan (anomalía de Sussman, aquí en versión suave). Los planificadores que tratan las metas como independientes producen planes subóptimos o fallan; las heurísticas de relajación subestiman precisamente estas interacciones.

📊 Propiedades y comparación

Criterio	Búsqueda atómica (013-016)	STRIPS/PDDL + heurística automática	GraphPlan
Representación del estado	caja negra	conjunto de literales	conjunto de literales
Heurística	diseñada a mano	extraída del dominio (h_FF, landmarks)	niveles del grafo (admisible)
Escala típica	juguete	dominios IPC con 10^20+ estados	intermedio (histórico)
Complejidad de decisión	depende del problema	PSPACE-completa	PSPACE-completa
Garantías	las del algoritmo (A* óptimo)	óptimo solo con heurística admisible	óptimo en pasos paralelos

⚠️ Errores conceptuales frecuentes

1. **"El plan es una política."** Un plan clásico es una secuencia abierta que presupone determinismo y observabilidad; si una acción falla, no dice qué hacer. Entornos inciertos exigen replanificación, planes con contingencias o MDPs (parte 02).
2. **Olvidar el supuesto de mundo cerrado.** En `(:init ...)` lo no declarado es falso. Omitir `(libre B)` no deja el hecho "desconocido": lo hace falso, y las acciones que lo requieren nunca serán aplicables.
3. **Confundir ADD/DEL con "todo el estado nuevo".** Los efectos describen solo *el cambio*; la suposición STRIPS conserva el resto. Escribir efectos que repiten hechos no cambiados es redundante; olvidar un DEL deja hechos fantasma (B "sigue" sobre la mesa después de apilarlo).
4. **Tratar las metas como independientes.** La anomalía de Sussman muestra que lograr metas una a una puede exigir deshacer trabajo; es la razón por la que la planificación no es una simple concatenación de búsquedas.
5. **"GraphPlan devuelve el plan óptimo en número de acciones."** Optimiza el número de *niveles paralelos*, no de acciones; y hoy su papel práctico es servir de heurística, no de planificador.

🚀 Del aprendizaje a la operación

La traza del ejemplo se verifica a mano; un despliegue real (logística, manufactura, operaciones espaciales — PDDL desciende del linaje que llevó a Remote Agent de la NASA) exige: modelar el dominio con expertos y validarlo contra el sistema físico (el error típico está en el modelo, no en el planificador), un ejecutor que monitorice precondiciones en tiempo de ejecución y replanifique al detectar divergencia, validación independiente de planes (herramientas tipo VAL), y manejo de tiempo, recursos y costos que el STRIPS puro

no expresa (PDDL2.1+, scheduling). El planificador es la pieza fácil de descargar; el modelo del dominio y el ciclo ejecutar-monitorizar-replanificar son el trabajo de ingeniería.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("workflow")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Russell & Norvig, *Artificial Intelligence: A Modern Approach* 4e — cap. 11 (Automated Planning) — uso: desarrollo extendido del tema
- Fikes & Nilsson (1971). "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving". *Artificial Intelligence* 2(3-4) — uso: fuente primaria del mecanismo estudiado
- Blum & Furst (1997). "Fast Planning Through Planning Graph Analysis". *Artificial Intelligence* 90(1-2) — uso: fuente primaria del mecanismo estudiado
- planning.wiki — referencia comunitaria de PDDL y planificadores — uso: referencia consultada en su fuente original
- Referencia de sintaxis PDDL — uso: referencia consultada en su fuente original
- Fast Downward — planificador de referencia (documentación oficial) — uso: referencia consultada en su fuente original

---

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P68 · STRIPS: un nuevo enfoque para aplicar la demostración de teoremas a la resolución de problemas	1971	Da a la planificación su representación duradera — precondición, añadir, borrar— y con ella una respuesta práctica al problema del marco.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · cap. 11 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define planificación clásica con strips y pddl sin usar una marca o framework como definición.
2. Explica la relación entre STRIPS, PDDL, precondiciones, efectos.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.
- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.

---

024 — Proyecto: asistente neuro-simbólico explicable

Parte: 01 — IA simbólica, búsqueda, lógica y planificación
Nivel: fundamentos · **Horas estimadas:** 10
Laboratorio: `capstone` · **Estado:** `EXECUTABLE_CORE`

🎯 Propósito

Comprender **proyecto: asistente neuro-simbólico explicable** dentro de la evolución de la inteligencia artificial, implementar un experimento mínimo verificable y distinguir qué parte constituye evidencia frente a una afirmación todavía no comprobada.

📚 Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: asistente neuro-simbólico explicable usando los conceptos `símbolos`, `reglas`, `explicación`, `integración`.
2. Ejecutar el laboratorio con una semilla explícita y revisar su contrato JSON.
3. Identificar al menos un supuesto, una limitación y un riesgo de aplicación.
4. Comparar el enfoque con la etapa anterior de la ruta de aprendizaje.
5. Producir una evidencia reproducible y una conclusión que no exceda los datos.

🧩 Conceptos centrales

`símbolos`, `reglas`, `explicación`, `integración`

🗺️ Ubicación en el mapa de la IA

Este proyecto cierra la parte simbólica del curso integrando todo lo anterior — búsqueda, lógica, reglas y planificación — con la pregunta que domina la IA actual: ¿cómo combinar el razonamiento simbólico (explicable, verificable, pobre en percepción) con el aprendizaje estadístico (perceptivo, robusto al ruido, opaco)? La IA **neuro-simbólica** es la respuesta programática: sistemas donde componentes neuronales y simbólicos cooperan. Ejemplos reales del patrón: AlphaGo (red neuronal guiando una búsqueda en árbol) y AlphaGeometry (modelo de lenguaje proponiendo construcciones que un motor simbólico verifica). Las partes 03-08 del curso desarrollan el lado neuronal; este proyecto fija el contrato del lado simbólico y de la explicación.

📖 Fundamentos

⚖️ Por qué integrar: fortalezas complementarias

Dimensión	Simbólico (reglas, lógica, planificación)	Neuronal (aprendizaje estadístico)
Origen del conocimiento	escrito por expertos	aprendido de datos
Percepción (imagen, audio, texto libre)	muy débil	fortaleza principal
Explicación	traza exacta de inferencia	post-hoc, aproximada
Garantías / verificación	posibles (correctitud, completitud)	raras, empíricas
Robustez al ruido	frágil (todo o nada)	degradación suave
Generalización composicional	fuerte (variables, cuantificadores)	límite conocido

La tesis neuro-simbólica: dejar la **percepción y la propuesta** al lado neuronal y la **verificación, el razonamiento y la explicación** al lado simbólico.

🧭 La taxonomía de Kautz

Henry Kautz (conferencia Engelmore, AAAI 2020; publicada en *AI Magazine*, 2022) clasificó las arquitecturas neuro-simbólicas en seis tipos, hoy la referencia estándar para *nombrar* un diseño:

```
Tipo 1  symbolic Neuro symbolic   — entrada y salida son símbolos; lo neuronal media
                                    (un LLM estándar visto como sistema NS mínimo)
Tipo 2  Symbolic[Neuro]           — sistema simbólico con subrutina neuronal interna
                                    (AlphaGo: MCTS simbólico llama a la red de valor)
Tipo 3  Neuro; Symbolic           — pipeline: la red extrae símbolos, el razonador
                                    opera sobre ellos (percepción → reglas)
Tipo 4  Neuro: Symbolic → Neuro   — conocimiento simbólico compilado en el
                                    entrenamiento de la red (p. ej. como datos/currículo)
Tipo 5  Neuro_{Symbolic}          — reglas tensorizadas como sesgo estructural dentro
                                    de la red (p. ej. Logic Tensor Networks)
Tipo 6  Neuro[Symbolic]           — razonamiento combinatorio verdadero embebido
                                    dentro del motor neuronal (aspiracional)
```

El asistente de este proyecto es **Tipo 3 (Neuro; Symbolic)**: un componente de recuperación/percepción produce hechos con confianza, y un motor de reglas (clase 022) razona sobre ellos y genera la explicación.

🔍 Explicabilidad: fiel vs. post-hoc

Distinción central del proyecto:

- **Explicación fiel (por diseño)**: la traza de reglas disparadas ES el cómputo que produjo la decisión. Auditable paso a paso; es lo que entrega el componente simbólico.
- **Explicación post-hoc**: una racionalización generada después sobre un modelo opaco (saliencia, LIME/SHAP, o un LLM "explicando" su respuesta). Puede ser plausible sin ser fiel.

Regla de diseño del asistente: **toda decisión que cruza el gate debe tener explicación fiel**; las partes neuronales aportan evidencia con confianza declarada, nunca la decisión final sin traza.

🏗️ Arquitectura del asistente del proyecto

El laboratorio `capstone` integra tres contratos ya vistos y añade el patrón de gobernanza:

1. **Recuperación** (`retrieval`): ranking de documentos por similitud — el sustituto didáctico del componente neuronal perceptivo. Produce evidencia puntuada, no verdades.
2. **Agente** (`agent`): ejecuta un plan de llamadas a herramientas conservando `action → observation` — la traza operativa.
3. **Política de seguridad** (`safety`): reglas explícitas que permiten o deniegan acciones con razones inspeccionables — el componente simbólico normativo.
4. **Gate final**: `release_gate: human_review_required`. La explicación no elimina la revisión humana; la hace posible.

```
percepción/recuperación (puntuada)  →  hechos con confianza
hechos + reglas de dominio          →  conclusiones con traza (encadenamiento, clase 022)
conclusiones + política             →  decisión permitir/denegar con razones
decisión + traza completa           →  explicación fiel + gate de revisión humana
```

🍼 Qué debe demostrar el proyecto

Criterio de aceptación conceptual: para cada salida del asistente debes poder responder, señalando datos del JSON, (a) *qué evidencia* entró, (b) *qué regla o paso* produjo cada conclusión, (c) *por qué* la política permitió o denegó, y (d) *qué NO* está garantizado (las `limitations`). Si alguna respuesta requiere "confiar en el modelo", ese eslabón no es explicable y hay que rediseñarlo o declararlo.

🧮 Ejemplo trabajado

Pipeline Tipo 3 en miniatura, con números. El módulo perceptivo (aquí: recuperación bag-of-words) puntúa documentos para la consulta `"herramientas estado objetivo"`:

```
score(agents) ≈ 0,87   score(models) ≈ 0,33   score(skills) ≈ 0,17
```

Reglas del asistente (umbral fijado por diseño, no aprendido):

```
R1: score(d) ≥ 0,8                       → evidencia_fuerte(d)
R2: evidencia_fuerte(d) ∧ permitido(read) → citar(d)
R3: acción ∉ permisos                    → denegar(acción, "tool_not_allowed")
```

Traza: R1 dispara solo para `agents` (0,87 ≥ 0,8); R2 autoriza citarlo porque la política permite `read`; una petición de `publish` es denegada por R3 con razón registrada. La explicación final es la cadena completa: *"cito `agents` porque su score 0,87 superó el umbral 0,8 (R1) y la política permite leer (R2); no publiqué porque `publish` no está en los permisos (R3)"*. Cada eslabón es verificable; el único componente no explicable (el score) queda **declarado como evidencia puntuada**, no como verdad.

📊 Propiedades y comparación

Criterio	Solo simbólico (022-023)	Solo neuronal (partes 04-06)	Neuro-simbólico Tipo 3 (este proyecto)
Percepción / texto libre	no maneja	fortaleza	delegada al módulo neuronal
Explicación de la decisión	fiel por diseño	post-hoc	fiel desde los hechos hacia adelante
Punto ciego	adquisición de conocimiento	opacidad, alucinación	calidad de los símbolos extraídos
Falla típica	regla ausente → silencio	error confiado → difícil de detectar	símbolo mal extraído → razonamiento correcto sobre premisa falsa
Verificación	formal posible	empírica	formal aguas abajo del extractor

⚠️ Errores conceptuales frecuentes

1. **"Neuro-simbólico = poner un LLM y pedirle que explique."** La autoexplicación de un modelo opaco es post-hoc: puede ser convincente y falsa. La explicación fiel exige que la traza sea el cómputo real.
2. **Asumir que el razonamiento correcto garantiza conclusiones correctas.** Si el módulo perceptivo extrae un símbolo erróneo, el motor de reglas razonará impecablemente sobre una premisa falsa. La calidad del sistema está acotada por la interfaz neuro→simbólica.
3. **Tratar los scores como probabilidades calibradas.** Un score de similitud de 0,87 no significa "87 % de probabilidad de relevancia"; los umbrales de las reglas deben fijarse con datos de validación, y esa calibración es parte del proyecto, no un detalle.
4. **Clasificar el diseño en la taxonomía por moda y no por flujo de datos.** El tipo de Kautz se determina por *quién llama a quién y qué cruza la interfaz* (símbolos, tensores, gradientes), no por qué componentes contiene.
5. **"La explicación elimina la necesidad de revisión humana."** Es al revés: la explicación fiel es lo que hace *posible* una revisión humana efectiva. El gate final no es un adorno del laboratorio; es el patrón de despliegue.

🚀 Del aprendizaje a la operación

El capstone integra tres funciones locales deterministas; un asistente neuro-simbólico real sustituye el retrieval por embeddings y un LLM (con lo que la interfaz neuro→simbólica se vuelve el punto crítico a evaluar: precisión de extracción de hechos, calibración de confianzas), añade persistencia y autenticación,

registra las trazas de explicación como telemetría auditable (parte de observabilidad, clase 168+), somete la base de reglas al ciclo de vida de la clase 022 (versionado, regresión) y define SLOs para el circuito humano del gate — quién revisa, en cuánto tiempo, con qué criterios. Nada de eso existe aquí, y el JSON del laboratorio lo declara en `limitations`: esa honestidad es exactamente el hábito que el proyecto evalúa.

🧪 Laboratorio

```
python lab.py
```

El laboratorio llama a `ai_evolution.labs.run_lab("capstone")`. Esta decisión evita una implementación divergente por clase: cada una conserva un entrypoint propio, pero los motores didácticos se prueban como una biblioteca común.

🔍 Evidencia esperada

- tipo de laboratorio y semilla;
- entradas o decisiones observables;
- resultado estructurado;
- lista `evidence` con hechos que pueden inspeccionarse;
- lista `limitations` que impide presentar la demo como producción.

📓 Notebooks

- 📓 `notebook.ipynb`: recorrido guiado con la materia resumida.
- ✍️ `notebook_student.ipynb`: ejercicios para resolver.
- ✅ `notebook_solution.ipynb`: solución de referencia explicada.

📝 Evaluación

Criterio	Peso
Comprensión conceptual	25 %
Ejecución reproducible	25 %
Interpretación basada en evidencia	25 %
Riesgos, límites y mejora propuesta	25 %

Consulta assessment.md para preguntas y criterio de aceptación.

⚠️ Errores comunes

Síntoma	Causa probable	Corrección
El código corre, pero no hay conclusión	Se confundió ejecución con aprendizaje	Explica qué demuestra y qué no demuestra
El resultado cambia sin explicación	No se registró semilla o configuración	Conserva semilla, versión y parámetros
Se promete uso real	Se extrapoló desde una demo educativa	Declara entorno, datos, límites y revisión humana
Se copia una métrica aislada	No existe baseline ni costo de error	Añade comparación y criterio de decisión

❓ Preguntas frecuentes

¿Debo usar una API comercial?
No. El núcleo funciona localmente. Las extensiones LIVE se documentan por separado.

¿El laboratorio representa una implementación industrial?
No por sí solo. Enseña el contrato y el patrón; producción exige integración, seguridad, observabilidad, pruebas y operación.

¿Dónde profundizo?
Revisa las especializaciones enlazadas en el README raíz y la ruta siguiente.

🔗 Referencias

- Kautz, H. (2022). "The third AI summer: AAAI Robert S. Engelmore Memorial Lecture". *AI Magazine* 43(1) — la taxonomía de sistemas neuro-simbólicos — uso: fuente primaria del mecanismo estudiado
- Garcez & Lamb (2020). "Neurosymbolic AI: The 3rd Wave" (arXiv:2012.05876) — uso: fuente primaria del mecanismo estudiado
- Marcus, G. (2020). "The Next Decade in AI: Four Steps Towards Robust Artificial Intelligence" (arXiv:2002.06177) — uso: fuente primaria del mecanismo estudiado
- Russell & Norvig, *Artificial Intelligence: A Modern Approach* 4e — caps. 7-11 (los componentes simbólicos integrados aquí) — uso: desarrollo extendido del tema
- Silver et al. (2016). "Mastering the game of Go with deep neural networks and tree search". *Nature* 529 — ejemplo canónico Symbolic[Neuro]

📜 Papers que fundamentan esta clase

> Bloque generado por `python scripts/link_papers_to_classes.py`. La fuente es `papers/catalog/papers.json`.

Paper	Año	Qué desbloqueó	Miniatura
P72 · IA neuro-simbólica: la tercera ola	2020	Ordena la agenda de integrar aprendizaje y razonamiento en vez de elegir uno de los dos.	notebook

Cada ficha explica el problema anterior, la matemática mínima, los límites y los errores de atribución más frecuentes. Para leerlas con método: cómo leer un paper de IA · anexos matemáticos.

📚 Bibliografía de apoyo

> Bloque generado por `python scripts/link_sources_to_classes.py`. Cada obra lleva su localizador verificado en `sources/bibliography.json`.

Los papers dicen **de dónde salió** el mecanismo. Estas obras lo **desarrollan** con el espacio que una clase no tiene: teoría completa, demostraciones y ejercicios.

Obra	Edición	Localizador	Papel en esta clase
Russell, Stuart J. y Norvig, Peter — *Artificial Intelligence: A Modern Approach*	4.ª · 2020	ISBN 9780134610993 · web de la obra	citada en las referencias de esta clase · caps. 7-11 · obra de referencia de la parte 01
Nilsson, N. J. — *Principles of Artificial Intelligence*	1980	ISBN 9780387113401	obra de referencia de la parte 01 · representación por espacios de estados

📝 Evaluación completa

❓ Preguntas

1. Define proyecto: asistente neuro-simbólico explicable sin usar una marca o framework como definición.
2. Explica la relación entre símbolos, reglas, explicación, integración.
3. Ejecuta `lab.py` dos veces con la misma semilla. ¿Qué debe conservarse?
4. Identifica una afirmación permitida y una afirmación exagerada sobre el resultado.
5. Propón una prueba negativa o un caso límite.

🏆 Reto verificable

Amplía el resultado del laboratorio con una clave `student_extension` que incluya:

- el supuesto que estás probando;
- una medición o comprobación;
- la conclusión;
- una limitación.

✅ Criterio de aceptación

- [ ] `lab.py` termina con código 0.
- [ ] El resultado contiene `kind`, `seed`, `evidence` y `limitations`.
- [ ] La extensión no modifica el comportamiento de otras clases.

- [ ] La interpretación referencia datos impresos por el laboratorio.
- [ ] Se declara al menos un riesgo o condición de no uso.
